

0 vimrc	2	cross-product.cpp	10
vimrc	2	extend-li-chao-tree.cpp	10
1 Number Theory	2	li-chao-tree.cpp	10
Exgcd.cpp	2	max-man-dis.cpp	11
catalan.cpp	2	minimum-euc-dis.cpp	11
findPrime.cpp	2	pick-theorem.cpp	11
matrixMultiple.cpp	2	ray-casting.cpp	11
ntt.cpp	2	動態凸包.cpp	11
system-of-linear-eqn.cpp	3	多邊形重心計算.cpp	12
2 Tree	3	找三點組成最大圓.cpp	12
LCA.cpp	3	極角排序.cpp	12
MST.cpp	3	5 Data Structure	13
binary-lifing.cpp	4	Array Version of Linked List.cpp	13
find-centriod.cpp	4	Binary Index Tree.cpp	13
hld.cpp	4	Disjoint Set Union.cpp	13
small-to-large.cpp	4	Segment Tree.cpp	13
tree-diameter.cpp	4	persistence-seg-tree.cpp	13
3 Graph	5	splay-tree.cpp	14
2-sat.cpp	5	6 Not that important	14
BellmanFord.cpp	5	EPS.cpp	14
Dijkstra.cpp	5	Gauce Elimination.cpp	14
Edmond.cpp	5	Joseph.cpp	15
articulation-point.cpp	6	7 New	15
bridge.cpp	6	Convex_hull_trick.cpp	15
dinic.cpp	6	Ray_casting.cpp	15
dominator-tree.cpp	6	block_cut_tree.cpp	16
floyd-warshall.cpp	7	cross_dot_product.cpp	16
hierholzer.cpp	7	dec_string_to_bitset.cpp	16
hopcroft-karp.cpp	7	dominator_tree.cpp	16
kuhn.cpp	7	extra.cpp	17
mcmf.cpp	8	fft.cpp	17
scc.cpp	8	fwt.cpp	18
toposort.cpp	8	kmp-automaton.cpp	18
4 Geometry	8	li_chao_tree.cpp	18
Determine whether a polygon is convex.cpp	8	manacher.cpp	19
Rotating Calipers 求凸包直徑.cpp	9	shoelace.cpp	19
all solutions for Diophantine Equation in given		suffix_automaton.cpp	19
range.cpp	9	suffixarray_and_lcp.cpp	19
convex-hull.cpp	9	xor_basis.cpp	20
		zfunction_and_kmp.cpp	20

0 vimrc

vimrc

```

1 "variable
2 let mapleader = ','
3
4 "
5 set number
6 set relativenumber
7 set noexpandtab
8 set shiftwidth=4
9 set tabstop=4
10
11 inoremap kj <Esc>
12 inoremap {<CR> {<Left><CR><Esc>0
13 tnoremap kj <C-]><C-n>
14 nnoremap <leader>c :w<CR>:!g++ -std=c++20 % -o %:r &&
↪ ./%:r<CR>
15 nnoremap [ [
16 nnoremap ] ]

```

1 Number Theory

Exgcd.cpp

```

1 array<ll,2> exgcd(ll a, ll b){
2     if(b == 0) return array<ll,2>{1,0};
3     auto ans = exgcd(b, a % b);
4     return array<ll,2>{ans[1], ans[0] - a / b *
↪ ans[1]};
5 }
6

```

catalan.cpp

```

1 C_0=1
2 C_n = $\sum C_i * C_{n-1-i}$ = $\frac{1}{n+1} C_n^{2n}$

```

findPrime.cpp

```

1 constexpr ll mxN=1e6+1;
2 mobius[1]=etf[1]=1;
3 forn(i,2,mxN){
4     if(!prime[i])
↪ prime[i]=i,mobius[i]=-1,etf[i]=i-1,st[++sz]=i;
5     forn(j,1,sz+1){
6         if(i*st[j]>mxN) break;
7         prime[i*st[j]]=st[j];
8         if(i%st[j]==0){
9             mobius[i*st[j]]=0;
10            etf[i*st[j]]=etf[i]*st[j];
11            break;
12        }
13        else mobius[i*st[j]]=mobius[i]*-1,etf[i*st
↪ j]=etf[i]*(st[j]-1);
14    }
15 }

```

matrixMultiple.cpp

```

1 // mod 需外部定義; 快速幂用時記得單位矩陣
2 vector<vector<ll>> matrixMultiply(const
↪ vector<vector<ll>>& A, const vector<vector<ll>>& B) {
3     ll rowA = A.size();
4     ll colA = A[0].size();

```

```

5     ll colB = B[0].size();
6     vector<vector<ll>> C(rowA, vector<ll>(colB, 0));
7     for (ll i = 0; i < rowA; ++i) {
8         for (ll k = 0; k < colA; ++k) {
9             for (ll j = 0; j < colB; ++j) {
10                C[i][j] = (C[i][j] + A[i][k] * B[k][j]) %
↪ mod;
11            }
12        }
13    }
14    return C;
15 }

```

ntt.cpp

```

1 using ll=long long;
2 // 注意: 兩模數CRT, 結果是真值 mod 998244353*1004535809
↪ (~1.0028e18)
3 // 真實係數必須 < 該值 (值域1e9且長度>~1000就會爆),
↪ 否則需三模數NTT
4 // 長度上限 2^21 (n+m <= ~2e6): fft2 模數只有 2^21 階單位根,
↪ 超過會靜默出錯
5 class FFT{
6     public:
7         ll mod,root,root_inv,maX,M,M_inv;
8         FFT(){
9             FFT(ll a,ll b,ll c,ll d,ll e,ll
↪ f):mod(a),root(b),root_inv(c),maX(d),M
↪ (e),M_inv(f){
10                ll ans=1;
11                while(m){
12                    if(m&1) ans=(ans*a)%mod;
13                    a=(a*a)%mod,m>>=1;
14                }
15                return ans;
16            }
17            void fft(ll sz,vc<ll> &arr,ll inv){
18                for(ll i=1,j=0;i<sz;++i){
19                    ll bit=sz>>1;
20                    ↪ for(;j&bit;bit>>=1)
21                    ↪ j^=bit; j^=bit;
22                    if(i<j)
23                        ↪ swap(arr[i],arr[j]);
24                }
25                for(ll l=2;l<=sz;l<=<=1){
26                    ll wlen=inv?root_inv:root;
27                    for(ll i=l;i<maX;i<=<=1)
28                        ↪ wlen=(wlen*wlen)%mod;
29                    for(ll i=0;i<sz;i+=l){
30                        ll w=1;
31                        for(ll
↪ j=0;j<l/2;++j){
32                            ll x=arr[i
↪ +j],y=
↪ (w*arr
↪ [i+j+l
↪ /2])%m
↪ od;
33                            arr[i+j]=
↪ (x+y<mo
↪ d?x+y:
↪ x+y-mo
↪ d);
34                            arr[i+j+l/
↪ 2]=(x-
↪ y>=0?x
↪ -y:x-y
↪ +mod);
35                            w=(w*wlen)
↪ %mod;
36                        }
37                    }
38                }
39                if(inv){
40                    ll n_1=fsp(sz,mod-2);

```

```

38         forn(i,0,sz){
39             arr[i]=(arr[i]*n_17)
40             }
41         }
42     }
43 };
44 constexpr ll MM=1002772198720536577;
45 ll mul(ll a,ll b){
46     __int128 aa=a,bb=b,cc;
47     cc=(aa*bb)%MM;
48     return (ll)cc;
49 }
50 void solve(){
51     FFT fft1(998244353,15311432,469870224,1<<23,100453);
52     FFT fft2(1004535809,702606812,700146880,1<<21,998244353);
53     ll n,m;
54     cin >> n >> m;
55     vc<ll> a(n+1),b(m+1);
56     for(auto &v:a) cin >> v;
57     for(auto &v:b) cin >> v;
58     ll sz=1;
59     for(;sz<a.size()+b.size();sz<<=1);
60     a.resize(sz),b.resize(sz);
61     vc<ll> A=a,B=b;
62     fft1.fft(sz,a,0);
63     fft1.fft(sz,b,0);
64     forn(i,0,sz) a[i]=(a[i]*b[i])%fft1.mod;
65     fft1.fft(sz,a,1);
66
67     fft2.fft(sz,A,0);
68     fft2.fft(sz,B,0);
69     forn(i,0,sz) A[i]=(A[i]*B[i])%fft2.mod;
70     fft2.fft(sz,A,1);
71
72     forn(i,0,n+m+1) cout <<
73         (mul(a[i],fft1.M*fft1.M_inv) +
74         mul(A[i],fft2.M*fft2.M_inv))%MM << "
75         \n"[i==n+m];
76 }
77 int main()
78 {
79     fast_io;
80     int testcase;
81     // cin>>testcase;
82     testcase=1;
83     while(testcase--){
84         solve();
85         return 0;
86     }
87 }

```

system-of-linear-eqn.cpp

```

1 // mod mdl1 下高斯消去。無解輸出-1; 無窮多解「不會偵測」
2 // (自由變數=0給特解)
3 // 要偵測無窮多解: 消去結束後 pivot 數 l < m 即有自由變數 =>
4 // 無窮多解
5 // 輸入係數需在 [0,mdl1) 內 (負數要先轉正); inv
6 // 其實是快速冪, 模數寫死 mdl1
7 constexpr ll mxN=505;
8 ll n,m;
9 ll inv(ll a,ll m){ll ans; for(ans=1;m;(ans*(a%mdl1))%mdl1==1;ans++)
10     return ans;}
11 void solve(istream &cin){
12     cin>>n>>m;
13     vc<ll> res(m,0);
14     vc<vc<ll>> a(n,vc<ll>(m+1));
15     forn(i,0,n) forn(j,0,m+1) cin>>a[i][j];
16     ll l=0;
17     for(ll i=0;i<m&&l<n;++i){
18         if(!a[l][i]) forn(j,l+1,n)
19             if(a[j][i]){swap(a[j],a[l]); break;}
20         if(!a[l][i]) continue;
21     }
22 }

```

```

1 ll c=inv(a[l][i],mdl1-2);
2 for(auto&v:a[l]) v=(v*c)%mdl1;
3 forn(j,l+1,n){
4     ll co=a[j][i];
5     forn(k,i,m+1) a[j][k]=(a[j][k]-co
6         *a[l][k])%mdl1+mdl1)%mdl1;
7 }
8 l++;
9 }
10 forn(i,n-1,0){
11     ll j=0;
12     while(j<m+1 && !a[i][j]) j++;
13     if(j==a[i].size()) continue;
14     if(j+1==a[i].size()){cout << -1 << '\n';
15         return;}
16     ll u=a[i][j];
17     forn(k,j+1,m)
18         u=((u-a[i][k]*res[k])%mdl1+mdl1)%mdl1;
19     res[j]=u;
20 }
21 forn(i,0,m) cout << res[i] << " \n"[i==m-1];
22 }
23 int main()
24 {
25     fast_io;
26     ll testcase;
27     // cin>>testcase;
28     testcase=1;
29     while(testcase--){
30         solve(cin);
31         return 0;
32     }
33 }

```

2 Tree

LCA.cpp

```

1 constexpr ll mxN=2e5+1;
2 ll pa[mxN][20],d[mxN]={};
3 vc<ll> adj[mxN];
4 void dfs(ll u=1,ll p=0){d[u]=d[p]+1,pa[u][0]=p;
5     for(auto&v:adj[u]) if(v!=p) dfs(v,u);}
6 void init(ll n){ // 建好 adj 後呼叫一次
7     dfs();
8     forn(j,1,20) forn(i,1,n+1)
9         pa[i][j]=pa[pa[i][j-1]][j-1];
10 }
11 ll lca(ll u,ll v){
12     if(d[u]<d[v]) swap(u,v);
13     forn(i,19,0) if(d[u]-(1<<i)>=d[v]) u=pa[u][i];
14     if(u==v) return u;
15     forn(i,19,0) if(pa[u][i]!=pa[v][i])
16         u=pa[u][i],v=pa[v][i];
17     return pa[u][0];
18 }

```

MST.cpp

```

1 //kruskal
2 //vertex is 1 index
3 #define mls multiset
4 ll n,pa[mxN]; //memset(pa,-1,sizeof(pa));
5 mls<ar<ll,3>> adj;
6 ll find(ll x){return pa[x]<0?x:pa[x]=find(pa[x]);}
7 void un(ll x,ll y){
8     x=find(x),y=find(y); if(x==y) return;
9     if(-pa[x]>-pa[y]) swap(x,y); pa[y]+=pa[x],pa[x]=y;
10 }
11 kruskal(){ // 回傳 MST 總權重; 結束後 cnt<n-1 代表圖不連通
12     ll cnt=0,wi=0;

```

```

15 while(!adj.empty() && cnt<n-1){
16     auto [w,u,v]=*adj.begin(); adj.erase(adj.begin());
17     if(find(u)!=find(v)) wi+=w,un(u,v),cnt++;
18 }
19 return wi;
20 }

```

binary-lifing.cpp

```

1 constexpr ll mxN=2e5+1;
2 ll n,q,p[mxN][20]={0}; //深度<mxN<2^18, 20層夠(49MB->32MB);
   ↳ n更大要加層數
3 vc<ll> adj[mxN];
4 void solve(istream &cin){
5     cin>>n>>q;
6     forn(i,2,n+1) cin>>p[i][0];
7     forn(j,1,20) forn(i,1,n+1) p[i][j]=p[p[i][j-1]][j-1];
8     while(q--){
9         ll x,k; cin>>x>>k;
10        k=min(k,(ll)mxN); // 跳超過深度必到0,
   ↳ clamp避免高位bit被忽略
11        forn(i,0,20) if(k>>i&1) x=p[x][i];
12        cout << (x?x:-1) << '\n';
13    }
14 }

```

find-centriod.cpp

```

1 void dfs(ll u=1,ll p=0){
2     d[u]=1;
3     for(auto&v:adj[u]){
4         if(v==p)
5             continue;
6         dfs(v,u);
7         d[u]+=d[v];
8     }
9 }
10 ll dfs_solve(ll u=1,ll p=0){
11     for(auto&v:adj[u]){
12         if(v==p)
13             continue;
14         if(d[v]*2>n) return dfs_solve(v,u);
15     }
16     return u;
17 }

```

hld.cpp

```

1 constexpr ll mxN=2e5+1;
2 ll tin=0,in[mxN]={},t[mxN<<1];
3 ll n,q,a[mxN],d[mxN]={},e[mxN]={};
4 vc<ll> adj[mxN];
5 ll dfs(ll u=1,ll p=0){
6     ll mx=0,sub=1;
7     forn(i,0,adj[u].size()){
8         ll v=adj[u][i];
9         if(v!=p){d[v]=d[u]+1; ll val=dfs(v,u);
   ↳ sub+=val; if(val>mx) mx=val,e[u]=v;}
10        else swap(adj[u][i],adj[u][0]);
11    }
12    return sub;
13 }
14 void hld(ll u=1,ll p=1){
15     in[u]++;tin,t[in[u]+n-1]=a[u];
16     if(e[u]) hld(e[u],p);
17     forn(i,1,adj[u].size()){ll v=adj[u][i];
   ↳ if(e[u]!=v) hld(v,v);}
18     e[u]=p;
19 }
20 void pst(ll p,ll val){for(t[p+n-1]=val;p>>=1;t[p]=max(t[p],
   ↳ <<1],t[p<<1|1]));}

```

```

11 qry(ll l,ll r){
12     ll ans=0;
13     for(l+=n-1,r+=n-1;l<=r;l>=1,r>=1){if(l&1)
   ↳ ans=max(ans,t[l++]); if(r&1^1)
   ↳ ans=max(ans,t[r--]);}
14     return ans;
15 }
16 ll query(ll a,ll b){
17     ll ans=0;
18     for(;e[a]!=e[b];b=adj[e[b]][0]){
19         if(d[e[a]]>d[e[b]]) swap(a,b);
20         ans=max(ans,qry(in[e[b]],in[b]));
21     }
22     if(in[a]>in[b]) swap(a,b);
23     return max(ans,qry(in[a],in[b]));
24 }
25 void solve(istream &cin){
26     cin >> n >> q;
27     forn(i,1,n+1) cin >> a[i];
28     forn(i,0,n-1){ll u,v; cin >> u >> v;
   ↳ adj[u].emp(v),adj[v].emp(u);}
29     adj[1].emp(0);
30     dfs();
31     hld();
32     forr(i,n-1,1) t[i]=max(t[i<<1],t[i<<1|1]);
33     for(ll op,a,b;q--;){
34         cin >> op >> a >> b;
35         if(op==1) pst(in[a],b);
36         else cout << query(a,b) << ' ';
37     }
38 }

```

small-to-large.cpp

```

1 // 注意: 每層遞迴持有deque, 鏈狀樹會爆棧且記憶體~300MB,
   ↳ 深樹題慎用
2 constexpr ll mxN=2e5+1;
3 ll n,k,res=0;
4 vc<ll> adj[mxN];
5 deque<ll> dfs(ll u=1,ll p=0){
6     deque<ll> cur(1,{1}),tmp;
7     for(auto &v:adj[u]){
8         if(v!=p){
9             tmp=dfs(v,u),tmp.push_front(0);
10            if(tmp.size()>cur.size())
   ↳ swap(tmp,cur);
11            forn(i,0,tmp.size()) if(k-i>=0 &&
   ↳ k-i<cur.size())
   ↳ res+=cur[k-i]*tmp[i];
12            forn(i,0,tmp.size())
   ↳ cur[i]+=tmp[i];
13        }
14    }
15    return cur;
16 }
17 void solve(istream &cin){
18     cin >> n >> k;
19     forn(i,0,n-1){ll u,v; cin >> u >> v;
   ↳ adj[u].emp(v),adj[v].emp(u);}
20     dfs();
21     cout << res << '\n';
22 }

```

tree-diameter.cpp

```

1 #define emp emplace_back
2 constexpr ll mxN=2e5+1;
3 ll n,res=0;
4 vc<ll> adj[mxN],dp(mxN);
5
6 void dfs(ll u=1,ll p=0){dp[u]=0; for(auto&v:adj[u])
   ↳ if(v!=p) dfs(v,u),res=max(res,dp[u]+dp[v]+1),dp[u]=max
   ↳ (dp[u],dp[v]+1);}

```

```

8  int main()
9  {
10     cin>>n; forn(i,0,n-1){ll u,v; cin>>u>>v;
        ↪ adj[u].emp(v),adj[v].emp(u);}
11     dfs();
12     cout << res << '\n';
13 }

```

3 Graph

2-sat.cpp

```

1  // 遞迴深度可達 2m(=2e5), 注意 stack 限制
2  constexpr ll mxN=2e5+1;
3  ll sc=0,scc[mxN];
4  ll id=0,st[mxN],vs[mxN]{};
5  ll n,m,tin=0,dfn[mxN]{};low[mxN];
6  vc<ll> adj[mxN];
7  void tarjan(ll u){
8      dfn[u]=low[u]=++tin,st[++id]=u,vs[u]=1;
9      for(auto &v:adj[u]){
10         if(!dfn[v])
            ↪ tarjan(v),low[u]=min(low[u],low[v]);
        else if(vs[v]) low[u]=min(low[u],dfn[v]);
11     }
12     if(low[u]==dfn[u]){
13         sc++;
14         for(;id;){
15             scc[st[id]]=sc,vs[st[id]]=0;
16             id--;
17             if(st[id+1]==u) break;
18         }
19     }
20 }
21 }
22 void solve(istream &cin){
23     cin >> n >> m;
24     forn(i,1,n+1){
25         char c1,c2; ll x,y,X,Y; cin >> c1 >> x >>
            ↪ c2 >> y;
26         X=x+m,Y=y+m;
27         if(c1=='-') swap(x,X);
28         if(c2=='-') swap(y,Y);
29         adj[X].emp(y),adj[Y].emp(x);
30     }
31     forn(i,1,2*m+1) if(!dfn[i]) tarjan(i);
32     forn(i,1,m+1) if(scc[i]==scc[i+m]){cout <<
        ↪ "IMPOSSIBLE" << '\n'; return;}
33     forn(i,1,m+1) cout << (scc[i]>scc[i+m]?'-':'+') <<
        ↪ " \n"[i==m];
34 }

```

BellmanFord.cpp

```

1  //SPFA; 回傳false=有負環(僅偵測從start可達的);
    ↪ d[]需外部初始化INF, d[start]=0
2  bool spfa(){
3      vc<ll> vs(n+1,0); //1-indexed 要開 n+1
4      queue<ll> q; q.emplace(start);
5      while(!q.empty()){
6          ll u=q.front(); q.pop(),vs[u]=0;
7          for(auto&[v,w]:adj[u]){
8              if(d[v]>d[u]+w){
9                  d[v]=d[u]+w;
10                 if(!vs[v]){vs[v]=1,cnt[v]++; if(cnt[v]>n)
                    ↪ return false; q.emplace(v);}
11             }
12         }
13     }
14     return true;
15 }

```

Dijkstra.cpp

```

1  // 前置: res[]全設INF、起點寫死1(res[1]=0); lazy-deletion寫法
2  void djc(){
3      mls<pll> hp;
4      res[1]=0,hp.emplace(pll{res[1],1});
5      while(!hp.empty()){
6          auto [d,u]=*hp.bg(); hp.erase(hp.bg());
7          if(d>res[u]) continue;
8          for(auto&[v,w]:adj[u]{
9              if(w+res[u]<res[v]) res[v]=w+res[u]
                ↪ ,hp.emplace(pll{res[v],v});
10         }
11     }
12 }

```

Edmond.cpp

```

1  constexpr ll mxN=505;
2  ll n,m,adj[mxN][mxN]{};g[mxN][mxN]{}; //adj=capacity
3  vc<ll> pa;
4  ll bfs(){
5      queue<pll> q; q.emplace(pll{0,inf});
6      while(!q.empty()){
7          auto [u,flow]=q.front(); q.pop();
8          forn(i,0,n)
9              if(g[u][i] && adj[u][i] &&
                ↪ pa[i]==-1){
10                 pa[i]=u;
11                 if(i==n-1) return
                    ↪ min(flow,adj[u][i]);
12                 q.emplace(pll{i,min(flow,a
                    ↪ dj[u][i])});
13             }
14     }
15     return 0;
16 }
17 void solve(istream &cin){
18     cin>>n>>m;
19     pa.resize(n);
20     forn(i,1,m+1){ll u,v; cin>>u>>v,u--,v--;
21         ↪ adj[u][v]++,g[u][v]++,adj[v][u]++,g[v][u]++;}
22     ↪ //重邊容量/重數要累加
23     ll f=0;
24     fill(all(pa),-1),pa[0]=0;
25     for(ll nf;nf=bfs();f+=nf){
26         ll cur=n-1;
27         while(cur) adj[pa[cur]][cur]-=nf,adj[cur][
            ↪ pa[cur]]+=nf,cur=pa[cur];
28         fill(all(pa),-1),pa[0]=0;
29     }
30     vc<pll> res;
31     forn(i,0,n) forn(j,0,n) if(g[i][j] && pa[i]!=-1 &&
        ↪ pa[j]==-1) forn(_,0,g[i][j])
        ↪ res.emplace(pll{i,j}); //重邊每條都要列
32     cout << res.size() << '\n';
33     for(auto&[u,v]:res) cout << u+1 << ' ' << v+1 <<
        ↪ '\n';
34 }
35 int main()
36 {
37     fast_io;
38     ll testcase;
39     // cin>>testcase;
40     testcase=1;
41     while(testcase--){
42         solve(cin);
43     }
44 }

```

articulation-point.cpp

```

1 //undirected graph articulation point; root呼叫
  ↳ tarjan(1,0), 不連通圖每個分量都要跑
2 ll n,tim=0,art=0;
3 vc<ll> dfn(mxN,0),low(mxN);
4 void tarjan(ll u,ll p){
5     ll cnt=0; bool isArt=false;
6     dfn[u]=low[u]=++tim;
7     for(auto&v:adj[u]){
8         if(!dfn[v]){
9             tarjan(v,u);
10            low[u]=min(low[u],low[v]);
11            if(low[v]>=dfn[u] && p) isArt=true; //非root:
  ↳ 某子樹回不到u之上
12            cnt++;
13        }
14        else if(v!=p) low[u]=min(low[u],dfn[v]);
15    }
16    if(!p && cnt>1) isArt=true; //root: 至少兩個子樹
17    art+=isArt;
18 }

```

bridge.cpp

```

1 constexpr ll mxN=1e5+1;
2 ll n,m,dfn[mxN]={},low[mxN],tin=0;
3 vc<ll> adj[mxN];
4 vc<ar<ll,2>> res;
5 void tarjan(ll u=1,ll p=0){
6     dfn[u]=low[u]=++tin;
7     bool skip=0; //父邊只跳過一次,
  ↳ 重邊(平行邊)才不會被誤判成橋
8     for(auto &v:adj[u]){
9         if(v==p && !skip){skip=1; continue;}
10        if(dfn[v]) low[u]=min(low[u],dfn[v]);
11        else{
12            tarjan(v,u),low[u]=min(low[u],low[v]);
  ↳ v]);
13            if(low[v]>dfn[u])
  ↳ res.emp(ar<ll,2>{u,v});
14        }
15    }
16 }

```

dinic.cpp

```

1 // 多測時 edg/adj/sz 是全域不會自動重置, 要自己清
2 constexpr ll mxN=505;
3 class fe{
4     public:
5         ll u,v,cp,fw;
6         fe(){
7             fe(ll U,ll V,ll CP):u(U),v(V),cp(CP){fw=0;}
8         };
9         ll n,m,sz=0;
10        vc<fe> edg;
11        queue<ll> q;
12        vc<ll> adj[mxN],ptr,dep;
13        bool bfs(){
14            while(!q.empty()){
15                ll u=q.front(); q.pop();
16                for(auto&id:adj[u]){
17                    if(edg[id].cp==edg[id].fw ||
  ↳ dep[edg[id].v]!=-1) continue;
18                    dep[edg[id].v]=dep[u]+1,q.emplace(
  ↳ edg[id].v);
19                }
20            }
21            return dep[n-1]!=-1;
22        }
23        ll dinic(ll u,ll push){
24            if(!push) return 0;

```

```

        if(u==n-1) return push;
        for(ll &cid=ptr[u];cid<adj[u].size();++cid){
            ll id=adj[u][cid];
            ll v=edg[id].v;
            if(dep[v]!=dep[u]+1) continue;
            ll tr=dinic(v,min(push,edg[id].cp-edg[id].
  ↳ fw));
            if(!tr) continue;
            edg[id].fw+=tr,edg[id^1].fw-=tr;
            return tr;
        }
        return 0;
    }
    void solve(istream &cin){
        cin>>n>m;
        ptr.resize(n),dep.resize(n);
        forn(i,1,m+1){
            ll u,v,c; cin>>u>>v>>c,u--,v--;
            edg.emp(fe{u,v,c}),edg.emp(fe{v,u,0});
            adj[u].emp(sz),adj[v].emp(sz+1),sz+=2;
        }
        ll res=0;
        for(;;){
            ll flow;
            fill(all(dep),-1);
            q.emplace(0),dep[0]=0;
            if(!bfs()) break;
            fill(all(ptr),0);
            for(;flow=dinic(0,inf);res+=flow);
        }
        cout << res << '\n';
    }
    int main()
    {
        fast_io;
        ll testcase;
        // cin>>testcase;
        testcase=1;
        while(testcase--){
            solve(cin);
            return 0;
        }
    }
}

```

dominator-tree.cpp

```

1 constexpr ll mxN=1e5+1;
2 ll n,m,tim=0,id[mxN],sd[mxN],pa[mxN],f[mxN],lb[mxN],et[mxN]
  ↳ []{},ret[mxN];
3 vc<ll> adj[mxN],radj[mxN],t[mxN],st;
4 void dfs(ll u=1){
5     et[u]=++tim,ret[tim]=u;
6     id[tim]=sd[tim]=lb[tim]=pa[tim]=tim;
7     for(auto&v:adj[u]){
8         if(!et[v]) dfs(v),f[et[v]]=et[u];
9         radj[et[v]].emp(et[u]);
10    }
11 }
12 ll find(ll u,ll x=0){
13     if(u==pa[u]) return x?-1:u;
14     ll v=find(pa[u],x+1);
15     if(v<0) return u;
16     if(sd[lb[u]]>sd[lb[pa[u]]]) lb[u]=lb[pa[u]];
17     pa[u]=v;
18     return x?v:lb[u];
19 }
20 void built(){
21     vc<ll> arr[n+1];
22     forr(i,n,1){
23         for(auto&v:radj[i])
  ↳ sd[i]=min(sd[i],sd[find(v)]);
24         if(i>1) arr[sd[i]].emp(i);
25         for(auto&v:arr[i]){
26             ll x=find(v);
27             if(sd[x]==sd[v]) id[v]=sd[v];
28             else id[v]=x;
29         }

```

```

30         if(i>1) pa[i]=f[i];
31     }
32     forn(i,2,n+1){
33         if(id[i]!=sd[i]) id[i]=id[id[i]];
34         t[ret[i]].emp(ret[id[i]]),t[ret[id[i]]].emp
            ↪ p(ret[i]);
35     }
36 }
37 void dfs_ans(ll u=1,ll p=0){
38     st.emp(u);
39     if(u==n){
40         sort(all(st));
41         cout << st.size() << '\n';
42         for(auto&v:st) cout << v << ' '; cout <<
            ↪ '\n';
43         exit(0);
44     }
45     for(auto&v:t[u]){
46         if(v==p) continue;
47         dfs_ans(v,u);
48     }
49     st.pop_back();
50 }
51 void solve(istream &cin){
52     cin>>n>>m;
53     forn(i,1,m+1){
54         ll u,v; cin>>u>>v;
55         adj[u].emp(v);
56     }
57     dfs();
58     built();
59     dfs_ans();
60 }
61

```

floyd-warshall.cpp

```

1 // 先設 f[x][x]=0 其餘INF
   ↪ (INF用0x3f3f3f3f3f3f3f3f可安全相加)
2 for (ll k = 1; k <= n; k++) {
3     for (ll x = 1; x <= n; x++) {
4         for (ll y = 1; y <= n; y++) {
5             f[x][y] = min(f[x][y], f[x][k] + f[k][y]);
6         }
7     }
8 }

```

hierholzer.cpp

```

1 //debrujin sequence 特化版(n<=15), 不是通用歐拉迴路模板,
   ↪ 一般題要整段改寫
2 ll n;
3 string s;
4 vc<ll> adj[1<<14|1];
5 void dfs(ll u=0){
6     while(!adj[u].empty()){
7         ll v; v=adj[u].back(); adj[u].pop_back();
8         dfs(v);
9         s+=(v&1)?'1':'0';
10    }
11 }
12 void solve(istream &cin){
13     cin>>n;
14     if(n==1){cout << "01" << '\n'; return;}
15     forn(i,0,1<<(n-1)){
16         adj[i].emp((i<<1)&((1<<(n-1))-1));
17         adj[i].emp((i<<1|1)&((1<<(n-1))-1));
18     }
19     dfs();
20     forn(i,0,n-1) s+='0';
21     reverse(all(s));
22     cout << s << '\n';
23 }

```

hopcroft-karp.cpp

```

1 // mxN=501 => n,m<=500; 匹配數 0 時不會輸出任何匹配行
2 constexpr ll mxN=501;
3 ll n,m,k,mat[mxN<<1]{},d[mxN<<1];
4 vc<ll> adj[mxN];
5 ll bfs(){
6     queue<ll> q;
7     memset(d,-1,sizeof(d));
8     forn(i,1,n+1) if(!mat[i]) d[i]=0,q.emplace(i);
9     while(!q.empty()){
10         ll u=q.front(); q.pop();
11         if(~d[0]) return 1;
12         for(auto &v:adj[u]){
13             if(!~d[mat[v]]) d[mat[v]]=d[u]+1
                ↪ ,q.emplace(mat[v]);
14         }
15     }
16     return 0;
17 }
18 bool dfs(ll u){
19     if(!u) return true;
20     for(auto &v:adj[u]) if(d[mat[v]]==d[u]+1 &&
        ↪ dfs(mat[v])){mat[u]=v,mat[v]=u;return true;}
21     d[u]=-1;
22     return false;
23 }
24 void solve(istream &cin){
25     cin >> n >> m >> k;
26     forn(i,0,k){ll u,v; cin >> u >> v;
        ↪ adj[u].emp(n+v);}
27     ll res=0;
28     for(;bfs();) forn(i,1,n+1) if(!mat[i]) res+=dfs(i);
29     cout << res << '\n';
30     if(res) forn(i,1,n+1) if(mat[i]) cout << i << ' '
        ↪ << (mat[i]-n) << '\n';
31 }

```

kuhn.cpp

```

1 // base=500寫死 => 隱含 n,m<=500 (女生編號=base+j),
   ↪ 換題記得改
2 constexpr ll mxN=1001;
3 constexpr ll base=500;
4 ll n,m,k,vs1[mxN],vs2[mxN],ok[mxN][mxN]{};
5 vc<ll> adj[mxN];
6 bool dfs(ll u){
7     if(vs2[u]) return false;
8     vs2[u]=1;
9     for(auto&v:adj[u]) if(!vs1[v] ||
        ↪ dfs(vs1[v])){vs1[v]=u; return true;}
10     return false;
11 }
12 void solve(istream &cin){
13     cin >> n >> m >> k;
14     //[1,500] boy [500+1,500+500] girl
15     forn(i,0,k){
16         ll u,v; cin >> u >> v,v+=base;
17         if(!ok[u][v])
18             adj[u].emp(v),adj[v].emp(u),ok[u][v]
                ↪ v=1;
19     }
20     memset(vs1,0,sizeof(vs1));
21     forn(i,1,n+m+1){
22         memset(vs2,0,sizeof(vs2));
23         dfs(i);
24     }
25     vc<ar<ll,2>> res;
26     forn(i,1,n+1){
27         for(auto &v:adj[i]){
28             if(vs1[v]==i)
                ↪ res.emp(ar<ll,2>{i,v-base});
29         }
30     }

```



```

30     }
31     cout << res.size() << '\n';
32     for(auto&[u,v]:res) cout << u << ' ' << v << '\n';
33 }

```

mcmf.cpp

```

1 // SPFA版MCMF: 邊權必須非負(無負環偵測, 原圖有負權會死迴圈);
↪ 源1匯n寫死; 多測清 e/id/adj
2 class Edge{
3     public:
4         ll u,v,c,w;
5         Edge(){ }
6         Edge(ll u,ll v,ll c,ll
            ↪ w):u(u),v(v),c(c),w(w){ }
7 };
8 constexpr ll mxN=501;
9 ll n,m,k,id=0,vs[mxN]{},d[mxN],pa[mxN];
10 vc<Edge> e;
11 vc<ll> adj[mxN];
12 ll bfs(){
13     memset(pa,-1,8*(n+1)),memset(d,0x3f,sizeof(d));
14     queue<ll> q;
15     d[1]=0,vs[1]=1,q.emplace(1);
16     while(!q.empty()){
17         auto u=q.front(); q.pop(),vs[u]=0;
18         for(auto &eid:adj[u]){
19             auto [x,v,c,w]=e[eid];
20             if(c && d[u]+w<d[v]){
21                 pa[v]=eid,d[v]=d[u]+w;
22                 if(!vs[v])
                    ↪ vs[v]=1,q.emplace(v);
23             }
24         }
25     }
26     return d[n]<INF;
27 }
28 void solve(){
29     cin >> n >> m >> k;
30     forn(i,1,m+1){
31         ll u,v,c,w; cin >> u >> v >> c >> w;
32         e.emp(Edge{u,v,c,w}),e.emp(Edge{v,u,0,-w});
            ↪ ,adj[u].emp(id++),adj[v].emp(id++);
33     }
34     ll f=0,res=0;
35     for(ll nf,v;f<k && bfs();){
36         for(nf=k-f,v=n;v!=1;nf=min(nf,e[pa[v]].c),2)
            ↪ v=e[pa[v]].u;
37         for(v=n;v!=1;e[pa[v]].c-=nf,e[pa[v]^1].c+=3)
            ↪ nf,v=e[pa[v]].u;
38         res+=d[n]*nf,f+=nf;
39     }
40     cout << (f<k?-1:res) << '\n';
41 }

```

scc.cpp

```

1 // scc編號從0開始(2-sat.cpp裡的版本是先++再給, 兩者編號差1,
↪ 混用注意)
2 void dfs(ll u){
3     dfn[u]=low[u]=++tim,st[++id]=u,vs[u]=1;
4     for(auto &v:adj[u]){
5         if(!dfn[v])
            ↪ dfs(v),low[u]=min(low[u],low[v]);
6         else if(vs[v]) low[u]=min(low[u],dfn[v]);
7     }
8     if(low[u]==dfn[u]){
9         for(ll v=-1;v!=u;scc[st[id]]=sc,vs[st[id]]3)
            ↪ =0,v=st[id],id--);
10        sc++;
11    }
12 }

```

toposort.cpp

```

1 void topo_sort() {
2     int n, m;
3     cin >> n >> m;
4     vector<vector<int>>> graph(n+1);
5     for (int i = 0; i < m; i++) {
6         int a, b;
7         cin >> a >> b;
8         graph[a].push_back(b);
9     }
10    vector<int> rdu(n+1, 0);
11    for (auto &node : graph) {
12        for (auto &now : node) {
13            rdu[now]++;
14        }
15    }
16    queue<int> que;
17    for (int i = 1; i <= n; i++) {
18        if (rdu[i] == 0) que.push(i);
19    }
20    vector<int> topo;
21    while (!que.empty()) {
22        int top = que.front();
23        que.pop();
24        topo.push_back(top);
25        for (auto &i : graph[top]) {
26            rdu[i]--;
27            if (rdu[i] == 0) que.push(i);
28        }
29    }
30    if (topo.size() == n) {
31        for (auto &i : topo) cout << i << " ";
32    }
33    else
34        cout << "IMPOSSIBLE";
35 }

```

4 Geometry

Determine whether a polygon is convex.cpp

```

1 // 注意:
↪ 自交但每個轉向同號的多邊形(如五角星一筆畫)會被誤判convex,
// 嚴格判定需另檢查總轉角恰為±360度; 整數座標>~3e7時double
↪ cross有精度風險, 應改ll
#include <vector>
using namespace std;

2 struct Point {
3     double x, y;
4 };
5
6 double cross(const Point& a, const Point& b, const Point&
↪ c) {
7     // 計算向量 ab x bc
8     double x1 = b.x - a.x, y1 = b.y - a.y;
9     double x2 = c.x - b.x, y2 = c.y - b.y;
10    return x1 * y2 - y1 * x2;
11 }
12
13 bool isConvex(const vector<Point>& poly) {
14     int n = poly.size();
15     if (n < 3) return false;
16     int sign = 0;
17     for (int i = 0; i < n; ++i) {
18         double c = cross(poly[i], poly[(i + 1) % n],
            ↪ poly[(i + 2) % n]);
19         if (c != 0) {
20             if (sign == 0)
                sign = (c > 0) ? 1 : -1;
21             else if ((c > 0 && sign < 0) || (c < 0 && sign
                ↪ > 0))

```



```

27         return false;
28     }
29 }
30 return true;
31 }

```

Rotating Calipers 求凸包直徑.cpp

```

1 struct Point {
2     double x, y;
3 };
4
5 double dist2(const Point& a, const Point& b) {
6     double dx = a.x - b.x, dy = a.y - b.y;
7     return dx * dx + dy * dy;
8 }
9
10 // 假設 convex 是逆時針的「嚴格」凸包(無共線點/重複點,
11 // 否則 two-pointer 不保證正確)
12 // 座標 > 1e7 時 double 平方和超過 2^53 有精度風險,
13 // 應改用 ll 算 dist2 最後才 sqrt
14 double convexHullDiameter(const vector<Point>& convex) {
15     int n = convex.size();
16     if (n == 1) return 0;
17     if (n == 2) return sqrt(dist2(convex[0], convex[1]));
18     double res = 0;
19     for (int i = 0, j = 1; i < n; ++i) {
20         while (dist2(convex[i], convex[(j + 1) % n]) >
21             dist2(convex[i], convex[j]))
22             j = (j + 1) % n;
23         res = max(res, dist2(convex[i], convex[j]));
24     }
25     return sqrt(res);
26 }

```

all solutions for Diophantine Equation in given range.cpp

```

1 /*
2 1. General solution to the linear Diophantine equation:
3 a * x + b * y = c
4 If (x0, y0) is a particular solution and g = gcd(a, b):
5   x = x0 + k * (b / g)
6   y = y0 - k * (a / g)
7   for any integer k
8 2. Number of integer solutions (x, y) within bounds:
9   minx <= x <= maxx
10  miny <= y <= maxy
11 需 a != 0 且 b != 0; |a|, |b|, |c|, 範圍 ~1e9 內不溢位
12 */
13 ll gcd_ext(ll a, ll b, ll &x, ll &y) {
14     if (!b) { x = 1, y = 0; return a; }
15     ll x1, y1, g = gcd_ext(b, a % b, x1, y1);
16     x = y1, y = x1 - (a / b) * y1;
17     return g;
18 }
19 bool find_any_solution(ll a, ll b, ll c, ll &x0, ll &y0, ll &g) {
20     g = gcd_ext(abs(a), abs(b), x0, y0);
21     if (c % g) return false;
22     x0 *= c / g, y0 *= c / g;
23     if (a < 0) x0 = -x0;
24     if (b < 0) y0 = -y0;
25     return true;
26 }
27 void shift_solution(ll &x, ll &y, ll a, ll b, ll cnt) {
28     x += cnt * b;
29     y -= cnt * a;
30 }
31
32 ll find_all_solutions(ll a, ll b, ll c, ll minx, ll maxx, ll miny, ll maxy) {
33     ll x, y, g;

```

```

34 if (!find_any_solution(a, b, c, x, y, g))
35     return 0;
36 a /= g;
37 b /= g;
38
39 ll sign_a = a > 0 ? +1 : -1;
40 ll sign_b = b > 0 ? +1 : -1;
41
42 shift_solution(x, y, a, b, (minx - x) / b);
43 if (x < minx)
44     shift_solution(x, y, a, b, sign_b);
45 if (x > maxx)
46     return 0;
47 ll lx1 = x;
48
49 shift_solution(x, y, a, b, (maxx - x) / b);
50 if (x > maxx)
51     shift_solution(x, y, a, b, -sign_b);
52 ll rx1 = x;
53
54 shift_solution(x, y, a, b, -(miny - y) / a);
55 if (y < miny)
56     shift_solution(x, y, a, b, -sign_a);
57 if (y > maxy)
58     return 0;
59 ll lx2 = x;
60
61 shift_solution(x, y, a, b, -(maxy - y) / a);
62 if (y > maxy)
63     shift_solution(x, y, a, b, sign_a);
64 ll rx2 = x;
65
66 if (lx2 > rx2)
67     swap(lx2, rx2);
68 ll lx = max(lx1, lx2);
69 ll rx = min(rx1, rx2);
70
71 if (lx > rx)
72     return 0;
73 return (rx - lx) / abs(b) + 1;
74 }

```

convex-hull1.cpp

```

// 注意: 重複點會全部留在輸出裡, 用前 sort+unique 去重
// 共線邊上的點會保留 (非嚴格凸包); /座標 / <= 1e9 時 cross
// 不溢位
class Point {
public:
    ll x, y;
    Point() {}
    Point(ll x, ll y) : x(x), y(y) {}
    friend ll operator^(const Point a, const Point b) { return a.x * b.y - a.y * b.x; }
    friend ll operator*(const Point a, const Point b) { return a.x * b.x + a.y * b.y; }
    friend Point operator-(const Point a, const Point b) { return Point{a.x - b.x, a.y - b.y}; }
    friend ostream& operator>>(ostream &out, const Point &a) { out << a.x << ' ' << a.y << '\n'; return out; }
    friend ostream& operator<<(ostream &out, const Point &a) { out << a.x << ' ' << a.y << '\n'; return out; }
    friend bool operator<(const Point a, const Point b) { return a.x < b.x || (a.x == b.x && a.y < b.y); }
};
ll dis(Point a, Point b) {
    return (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y);
}

```

```

25 }
26 ll ok(Point a,Point b,Point c){
27     ll ang=(b-a)^(c-a); //ang>0 false ang< true
28     return ang<0;
29 }
30 constexpr ll mxN=2e5+5;
31 ll st[mxN],id=0;
32 void solve(istream &cin){
33     ll n; cin >> n;
34     vc<Point> a(n); for(auto &v:a) cin >> v;
35     forn(i,1,n) if(a[i]<a[0]) swap(a[i],a[0]);
36     sort(1+all(a),[p=a[0]](Point a,Point b){ll
        ↪ ang=(a-p)^(b-p); return
        ↪ !ang?dis(p,a)<dis(p,b):ang>0;});
37     {
38         ll j; for(j=n-1;j>0 &&
        ↪ ((a[j]-a[0])^(a[n-1]-a[0]))==0;j--);
        ↪ reverse(j+1+a.bg(),a.ed());
39     }
40     forn(i,0,n){
41         while(id>1 &&
        ↪ ok(a[st[id-1]],a[st[id]],a[i])) --id;
        ↪ st[++id]=i;
42     }
43     cout << id << '\n';
44     if(id) for(;id;cout << a[st[id--]]);
45 }
46

```

cross-product.cpp

```

1 class Point{
2     public:
3         ll x,y;
4         Point(){}
5         Point(ll x,ll y):x(x),y(y){}
6         ll f(Point a,Point b){
7             return a.x*b.y-b.x*a.y;
8         }//cross product
9         friend Point operator-(const Point a,const
        ↪ Point b){
10             return Point{a.x-b.x,a.y-b.y};
11         }
12         friend istream& operator>>(istream
        ↪ &in,Point &a){
13             in >> a.x >> a.y;
14             return in;
15         }
16 };

```

extend-li-chao-tree.cpp

```

1 // solve()斜率 (y2-y1)/(x2-x1): 垂直線段(x1==x2)會除以零,
  ↪ 不整除會截斷
2 class Line{
3     public:
4         ll m,c;
5         Line(){m=0,c=-inf;} //通用版: 空節點 ==-inf
6         //Line(){m=0,c=-1;} //原題版sentinel: 該題
        ↪ 「查無值輸出-1」時用這行
7         Line(ll m,ll c):m(m),c(c){}
8         ll cal(ll x){return m*x+c;}
9 };
10 constexpr ll mxN=1e5;
11 Line t[mxN<<2];
12 void insert(ll i,ll sl,ll sr,ll l,ll r,Line L){
13     if(l>sr || r<sl) return ;
14     ll mid=(sl+sr)>>1;
15     if(l<=sl && sr<=r){
16         if(L.cal(mid)>t[i].cal(mid)) swap(L,t[i]);
17         if(sl==sr) return;
18         if(L.m>t[i].m)
        ↪ insert(i<<1|1,mid+1,sr,l,r,L);
19         else insert(i<<1,sl,mid,l,r,L);
20         return ;

```

```

21 }
22 insert(i<<1,sl,mid,l,r,L);
23 insert(i<<1|1,mid+1,sr,l,r,L);
24 }
25 ll query(ll i,ll sl,ll sr,ll x){
26     ll mid=(sl+sr)>>1;
27     if(sl==sr) return t[i].cal(x);
28     if(x<=mid) return
        ↪ max(t[i].cal(x),query(i<<1,sl,mid,x));
        ↪ else return
        ↪ max(t[i].cal(x),query(i<<1|1,mid+1,sr,x));
29 }
30 void solve(istream &cin){
31     ll n,m,r0=-1; cin >> n >> m;
32     forn(i,0,n){
33         ll x1,x2,y1,y2; cin >> x1 >> y1 >> x2 >>
        ↪ y2;
34         if(!x1) r0=max(r0,y1);
35         Line l((y2-y1)/(x2-x1),-x1*(y2-y1)/(x2-x1)
        ↪ + y1);
        ↪ insert(1,1,mxN,x1,x2,1);
36     }
37     cout << r0 << " \n"[m==1];
38     forn(i,1,m+1) cout << query(1,1,mxN,i) << "
        ↪ \n"[i==m];
39 }
40

```

li-chao-tree.cpp

```

// solve()是特定題: (y2-y1)/m 整數除法, 斜率不整除會錯,
  ↪ 通用時只抄struct+insert+query
class Line{
    public:
        ll m,c;
        Line(){m=0,c=-inf;} //通用版: 空節點=-inf
        ↪ (m=0, cal不會溢位)
        //Line(){m=c=0;} //原題版:
        ↪ 答案保證>=0才可用, 否則負答案會被鎖成0
        Line(ll m,ll c):m(m),c(c){}
        ll cal(ll x){return m*x+c;}
};
constexpr ll mxN=1e5;
Line t[mxN<<2];
void insert(ll i,ll sl,ll sr,Line l){
    ll mid=(sl+sr)>>1;
    if(l.cal(mid)>t[i].cal(mid)) swap(l,t[i]);
    if(sl==sr) return;
    if(l.m>t[i].m) insert(i<<1|1,mid+1,sr,l);
    else insert(i<<1,sl,mid,l);
}
ll query(ll i,ll sl,ll sr,ll x){
    ll mid=(sl+sr)>>1;
    if(sl==sr) return t[i].cal(x);
    if(x<=mid) return
        ↪ max(t[i].cal(x),query(i<<1,sl,mid,x));
        ↪ else return
        ↪ max(t[i].cal(x),query(i<<1|1,mid+1,sr,x));
}
void solve(istream &cin){
    ll n,m; cin >> n >> m;
    forn(i,0,n){
        ll y1,y2; cin >> y1 >> y2;
        // (y2-y1)/(m-0) = (y-y1)/x
        //x * (y2-y1)/m + y1= y
        Line l((y2-y1)/m,y1);
        insert(1,1,mxN,1);
    }
    forn(i,0,m+1) cout << query(1,1,mxN,i) << "
        ↪ \n"[i==m];
}

```

max-man-dis.cpp

```

1 constexpr ll mxN=2e5+1;
2 ll n,d[2][2];
3 void solve(){
4     cin >> n;
5     d[0][0]=d[1][0]=-inf;
6     d[0][1]=d[1][1]=inf;
7     forn(i,0,n){
8         ll x,y;
9         cin >> x >> y;
10        d[0][0]=max(d[0][0],x+y);
11        d[0][1]=min(d[0][1],x+y);
12        d[1][0]=max(d[1][0],x-y);
13        d[1][1]=min(d[1][1],x-y);
14        cout <<
            ↪ max(d[0][0]-d[0][1],d[1][0]-d[1][1])
            ↪ << '\n';
15    }
16 }

```

minimum-euc-dis.cpp

```

1 class Point{
2     public:
3         ll x,y;
4         Point(){}
5         Point(ll x,ll y):x(x),y(y){}
6         friend ll operator^(const Point a,const
            ↪ Point b){return a.x*b.y-a.y*b.x;}
7         friend ll operator*(const Point a,const
            ↪ Point b){return a.x*b.x+a.y*b.y;}
8         friend Point operator-(const Point a,const
            ↪ Point b){
9             return Point{a.x-b.x,a.y-b.y};
10        }
11        friend istream& operator>>(istream
            ↪ &in,Point &a){
12            in >> a.x >> a.y;
13            return in;
14        }
15        friend bool operator<(const Point a,const
            ↪ Point b){return a.x<b.x||(a.x==b.x &&
            ↪ a.y<b.y);}
16 };
17 ll euc(Point a,Point b){
18     return (a.x-b.x)*(a.x-b.x)+(a.y-b.y)*(a.y-b.y);
19 }
20 void solve(istream &cin){
21     ll n; cin >> n;
22     vc<Point> a(n); for(auto &v:a) cin >> v;
23     sort(all(a),[](Point u,Point v){return
            ↪ (u.y<v.y)|| (u.y==v.y && u.x<v.x)});
24     ll d;
25     mls<Point> hp;
26     d=euc(a[0],a[1]),hp.emplace(a[0]),hp.emplace(a[1]);
27     for(ll i=2,p=0;i<n;i++){
28         for(;p<i && (a[p].y-a[i].y)*(a[p].y-a[i].y)
            ↪ >=d;hp.erase(hp.lwb(a[p])),p++);
29         ↪ //p<i! 寫p<n重複點(d=0)會清空set後eras
30         ↪ (end())炸掉
31         auto it=hp.lwb(a[i]);
32         auto front=it,back=it;
33         forn(j,0,4){
34             if(front!=hp.ed())
35                 ↪ d=min(d,euc(a[i],*front));
36             else break; front++;
37         }
38         forn(j,0,4){
39             if(back!=hp.ed())
40                 ↪ d=min(d,euc(a[i],*back));
41             if(back!=hp.bg()) back--; else
42                 ↪ break;
43         }
44         hp.emplace(a[i]);
45     }
46 }

```

```

        cout << d << '\n';
    }

```

pick-theorem.cpp

```

1 void solve(istream &cin){
2     ll n; cin >> n;
3     vc<Point> a(n);
4     for(auto &v:a) cin >> v;
5     ll area=0,B=0;
6     forn(i,0,n){
7         ll j=(i+1)%n;
8         area+=a[i]^a[j];
9         B+=gcd(abs(a[i].x-a[j].x),abs(a[i].y-a[j].y));
10    }
11    cout << (abs(area)/2+1-B/2) << ' ' << B << '\n';
12    //area=I+B/2-1 I=internal node B=boundary node
13 }

```

ray-casting.cpp

```

1 class Point{
2     public:
3         ll x,y;
4         Point(){}
5         Point(ll x,ll y):x(x),y(y){}
6         friend ll operator^(const Point a,const
            ↪ Point b){return a.x*b.y-a.y*b.x;}
7         friend ll operator*(const Point a,const
            ↪ Point b){return a.x*b.x+a.y*b.y;}
8         friend Point operator-(const Point a,const
            ↪ Point b){
9             return Point{a.x-b.x,a.y-b.y};
10        }
11        friend istream& operator>>(istream
            ↪ &in,Point &a){
12            in >> a.x >> a.y;
13            return in;
14        }
15    };
16 bool on(Point &a,Point &b,Point &c){
17     if((b-a)^(c-a)) return false;
18     return 0<=((b-a)*(c-a)) &&
            ↪ ((b-a)*(c-a))<=((b-a)*(b-a));
19 }
20 void query(vc<Point> &a,Point &p){
21     ll n=a.size(),ok=0;
22     forn(i,0,n){
23         ll j=(i+1)%n;
24         if(on(a[i],a[j],p)){cout << "BOUNDARY" <<
            ↪ '\n'; return;}
25         if(p.y>a[i].y!=p.y>a[j].y){
26             ll sg=(a[j]-a[i])^(p-a[i]);
27             if(sg<0==a[i].y>a[j].y) ok^=1;
28         }
29     }
30     cout << (ok&1?"INSIDE":"OUTSIDE") << '\n';
31 }

```

動態凸包.cpp

```

1 struct Point {
2     long long x, y;
3     bool operator<(const Point& p) const {
4         return x < p.x || (x == p.x && y < p.y);
5     }
6     bool operator==(const Point& p) const { return x ==
            ↪ p.x && y == p.y; }
7 };

```

```

8
9 long long cross(const Point& a, const Point& b, const
  ↪ Point& c) {
10     // (b-a) x (c-a)
11     return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x
  ↪ - a.x);
12 }
13
14 // 只支援插入; 嚴格凸包 (共線點會被移除), 每條殼同 x 只留一點
15 struct DynamicConvexHull {
16     set<Point> up, dn; // 上凸殼、下凸殼 (皆由左至右)
17
18     void insert(const Point& p) {
19         insertHull(up, p, +1);
20         insertHull(dn, p, -1);
21     }
22
23     // 逆時針凸包頂點 (不重複); 1 點/2 點退化也正確
24     vector<Point> getConvexHull() const {
25         vector<Point> res(dn.begin(), dn.end());
26         for (auto it = up.rbegin(); it != up.rend(); ++it)
27             if (!(res.size() && (*it == res.back() || *it
  ↪ == res.front()))
  ↪ res.push_back(*it);
28         return res;
29     }
30
31 private:
32     // side==+1 上殼, -1 下殼; p 在殼上或內側回傳 true
33     bool covered(const set<Point>& h, const Point& p, long
  ↪ side) const {
34         if (h.empty() || p.x < h.begin()->x || p.x >
  ↪ h.rbegin()->x) return false;
35         auto r = h.lower_bound(Point{p.x, LLONG_MIN}); //
  ↪ 第一個 x>=p.x
36         if (r->x == p.x) return side * (r->y - p.y) >= 0;
37         return side * cross(*prev(r), *r, p) <= 0;
38     }
39
40     void insertHull(set<Point>& h, const Point& p, long
  ↪ side) {
41         if (covered(h, p, side)) return; // 在殼內,
  ↪ 不影響凸包
42         auto r = h.lower_bound(Point{p.x, LLONG_MIN});
43         if (r != h.end() && r->x == p.x) h.erase(r); //
  ↪ 同x被p支配
44         auto it = h.insert(p).first;
45         while (it != h.begin()) { // 往左刪不再凸的點
46             auto b = prev(it);
47             if (b == h.begin()) break;
48             auto a = prev(b);
49             if (side * cross(*a, *b, *it) >= 0) h.erase(b);
50             else break;
51         }
52         while (next(it) != h.end()) { // 往右刪不再凸的點
53             auto b = next(it);
54             if (next(b) == h.end()) break;
55             auto c = next(b);
56             if (side * cross(*it, *b, *c) >= 0) h.erase(b);
57             else break;
58         }
59     }
60 };

```

多邊形重心計算.cpp

```

1 // 注意: 面積為0(退化/全共線)會除以零得NaN;
  ↪ 整數座標>-3e7時double精度風險(可改ll累加)
2 struct Point {
3     double x, y;
4 };
5
6 Point polygonCentroid(const vector<Point>& poly) {
7     double cx = 0, cy = 0, area = 0;
8     int n = poly.size();
9     for (int i = 0; i < n; ++i) {
10         double cross = poly[i].x * poly[(i + 1) % n].y -
  ↪ poly[(i + 1) % n].x * poly[i].y;

```

```

11         cx += (poly[i].x + poly[(i + 1) % n].x) * cross;
12         cy += (poly[i].y + poly[(i + 1) % n].y) * cross;
13         area += cross;
14     }
15     area /= 2.0;
16     cx /= (6.0 * area);
17     cy /= (6.0 * area);
18     return {cx, cy};
19 }

```

找三點組成最大圓.cpp

```

1 // 找三點使外接圓最大:  $O(n^3)$  枚舉 (答案的三點通常「不在」
  ↪ 凸包上, 不能旋轉卡殼)
2 // 共線三點視為無效 (回傳 0); 全部共線輸出 0.  $n \leq 300$  內可跑
3 #include <bits/stdc++.h>
4 using namespace std;
5 typedef long long ll;
6 typedef pair<ll, ll> P;
7
8 ll cross(P a, P b) { return a.first * b.second - a.second
  ↪ * b.first; }
9 ll dist2(P a, P b) {
10     ll dx = a.first - b.first, dy = a.second - b.second;
11     return dx * dx + dy * dy;
12 }
13 double dist(P a, P b) { return sqrt((double)dist2(a, b)); }
14
15 //  $R = abc / (4S)$ ; 2S 用 ll cross 算避免浮點誤判共線
16 double circumradius(P a, P b, P c) {
17     ll S2 = abs(cross({b.first-a.first, b.second-a.second},
  ↪ {c.first-a.first,
  ↪ c.second-a.second})); // 2S
18     if (S2 == 0) return 0; // 共線
19     return dist(b,c) * dist(a,c) * dist(a,b) / (2.0 *
  ↪ (double)S2);
20 }
21
22 int main() {
23     int n;
24     cin >> n;
25     vector<P> pts(n);
26     for (int i = 0; i < n; ++i) cin >> pts[i].first >>
  ↪ pts[i].second;
27     double ans = 0;
28     for (int i = 0; i < n; ++i)
29         for (int j = i + 1; j < n; ++j)
30             for (int k = j + 1; k < n; ++k)
31                 ans = max(ans, circumradius(pts[i],
  ↪ pts[j], pts[k]));
32     cout << fixed << setprecision(6) << ans << endl;
33 }

```

極角排序.cpp

```

1 struct Point {
2     long long x, y;
3 };
4
5 // 以 base 為原點極角排序: 從+x軸(角度0)逆時針到2;
  ↪ 同角度距離近的在前
6 // 必須先分上下半平面再比cross,
  ↪ 否則超過180度時comparator無效(UB)
7 // base 本身若在點集中會被排在最前 (視為角度 0 距離 0)
8 void polarSort(vector<Point>& points, Point base) {
9     auto half = [&](const Point& p) { // 0:上半平面(含+x軸),
  ↪ 1:下半平面(含-x軸)
10         long long dx = p.x - base.x, dy = p.y - base.y;
11         return (dy < 0 || (dy == 0 && dx < 0)) ? 1 : 0;
12     };
13     sort(points.begin(), points.end(), [&](const Point& a,
  ↪ const Point& b) {
14         if (half(a) != half(b)) return half(a) < half(b);

```

```

15     long long ax = a.x - base.x, ay = a.y - base.y;
16     long long bx = b.x - base.x, by = b.y - base.y;
17     long long cr = ax * by - ay * bx;
18     if (cr != 0) return cr > 0;
19     return ax * ax + ay * ay < bx * bx + by * by; //
    ↪ 座標<=1e9不溢位
20 });
21 }

```

5 Data Structure

Array Version of Linked List.cpp

```

1 // 陣列版雙向串列. 用前必須先 init 串好(0 與 n+1 是 sentinel
    ↪ 資料放 1..n),
2 // 否則 cut/insert 解參考 NULL 直接 segfault. cut(p): 把 p
    ↪ 從串列拔出並回傳;
3 // insert(p,q): 把 q 插到 p 右邊. sentinel 本身不可被 cut.
4 struct node{
5     struct node *nxt, *prv;
6     long long v;
7 }pos[100002];
8
9 void init(int n){ // 0 -> 1 -> ... -> n -> n+1
10     for(int i=0;i<=n+1;i++){
11         pos[i].nxt = (i<=n ? &pos[i+1] : NULL);
12         pos[i].prv = (i>=1 ? &pos[i-1] : NULL);
13     }
14 }
15
16 inline node* cut(node *p) // cut the node
17 {
18     p->nxt->prv = p->prv;
19     p->prv->nxt = p->nxt;
20     p->nxt = p->prv = NULL;
21     return p;
22 }
23
24 inline node* insert(node *p, node *q) // insert node q to
    ↪ right side of node p
25 {
26     q->nxt = p->nxt;
27     p->nxt->prv = q;
28     q->prv = p;
29     p->nxt = q;
30     return q;
31 }

```

Binary Index Tree.cpp

```

1 //BIT is always 1 index; built是O(mn)且用+=累加,
    ↪ 多測要先清t[]並保證a[n+1..]為0
2 ll n,t[mxN],a[mxN];
3 void built(){for(i,1,mxN){ t[i]+=a[i]; if(i+(i&-i)<mxN)
    ↪ t[i+(i&-i)]+=t[i];}} //O(n)
4 void pad(ll p,ll val){for(;p<mxN;t[p]+=val,p+=p&-p);}
5 ll query(ll p){ll ans=0; for(;p;ans+=t[p],p=p&-p); return
    ↪ ans;}

```

Disjoint Set Union.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class DisjointSets {
5     private:
6         vector<int> parents;
7         vector<int> sizes;
8
9     public:

```

```

DisjointSets(int size) : parents(size),
    ↪ sizes(size, 1) {
        for (int i = 0; i < size; i++) {
            ↪ parents[i] = i; }
    }

    /** @return the "representative" node in x's
    ↪ component */
    int find(int x) {
        return parents[x] == x ? x : (parents[x] =
            ↪ find(parents[x]));
    }

    /** @return whether the merge changed connectivity
    ↪ */
    bool unite(int x, int y) {
        int x_root = find(x);
        int y_root = find(y);
        if (x_root == y_root) { return false; }

        if (sizes[x_root] < sizes[y_root]) {
            ↪ swap(x_root, y_root); }
        sizes[x_root] += sizes[y_root];
        parents[y_root] = x_root;
        return true;
    }

    /** @return whether x and y are in the same
    ↪ connected component */
    bool connected(int x, int y) { return find(x) ==
        ↪ find(y); }
};

```

Segment Tree.cpp

```

1 //range add range query // sum query
2 //初始化(1-indexed): a[i]放進 t[n+1+i] (i=1..n)
    ↪ 再呼叫無參數built(); 多測要清t/lz
3 #define forr(a,b,c) for(ll a=b;a>=c;--a)
4 #define forn(a,b,c) for(ll a=b;a<=c;++a)
5 constexpr ll mxN=2e5+1;
6 #pragma GCC target("lzcnt")
7 ll n,q,t[mxN<<1],lz[mxN];
8 void built(){forr(i,n-1,1)t[i]=t[i<<1]+t[i<<1|1];}
9 void apply(ll p,ll k,ll val){t[p]+=val*k; if(p<n)
    ↪ lz[p]+=val;}
10 void push(ll p){for(ll h=63-__builtin_clzll(p);h;h--)
    ↪ if(lz[p>>h]){apply((p>>h)<<1,1<<(h-1),lz[p>>h]),apply(
    ↪ (p>>h)<<1|1,1<<(h-1),lz[p>>h]),lz[p>>h]=0;}}
11 void built(ll p){for(;p>=1;){if(!lz[p])
    ↪ t[p]=t[p<<1]+t[p<<1|1];}}
12 void rad(ll l,ll r,ll val){
13     ll l0=l+n-1,r0=r+n-1,k=1; push(l0),push(r0);
14     for(;l<=r;l>=1,r>=1,k<=1){if(l&1) apply(l++,k,val);
    ↪ if(r&1) apply(r--,k,val);}
    built(l0),built(r0);
15 }
16
17 ll query(ll l,ll r){ ll ans=0; push(l+=n-1),push(r+=n-1);
    ↪ for(;l<=r;l>=1,r>=1){if(l&1) ans+=t[l++]; if(r&1)
    ↪ ans+=t[r--];} return ans;} //range query
18 ll query(ll p){push(p+=n-1); return t[p];} // point query

```

persistence-seg-tree.cpp

```

1 // 節點數: build用2n-1, 每次單點改約log(n)+1個; mxT要 >=
    ↪ 2n+q*20 (n=q=2e5時約4.2e6)
2 // 記憶體約 7e6*(8+4+4) 106MB; 多測時 t/lc/rc 殘留值會錯,
    ↪ 需清到 sz 為止
3 constexpr ll mxN=2e5+1;
4 constexpr ll mxT=7e6+1;
5 ll n,q,t[mxT],a[mxN],rt[mxN<<1],rsc,sz; int
    ↪ lc[mxT],rc[mxT];
6 void built(ll i,ll sl,ll sr){

```

```

7         if(s1==sr){t[i]=a[s1]; return;}
8         ll mid; mid=(s1+sr)>>1;
9         lc[i]++sz,rc[i]++sz;
10        built(lc[i],s1,mid);
11        built(rc[i],mid+1,sr);
12        t[i]=t[lc[i]]+t[rc[i]];
13    }
14    ll pst(ll pos,ll val,ll i,ll s1,ll sr){
15        ll x; x=++sz;
16        if(s1==sr){t[x]=val; return x;}
17        ll mid; mid=(s1+sr)>>1;
18        if(pos<=mid)
19            ↪ lc[x]=pst(pos,val,lc[i],s1,mid),rc[x]=rc[i];
20        else lc[x]=lc[i],rc[x]=pst(pos,val,rc[i],mid+1,sr);
21        t[x]=t[lc[x]]+t[rc[x]];
22        return x;
23    }
24    ll query(ll l,ll r,ll i,ll s1,ll sr){
25        if(l>sr || r<s1) return 0;
26        if(l<=s1 && sr<=r) return t[i];
27        ll mid; mid=(s1+sr)>>1;
28        return query(l,r,lc[i],s1,mid)+query(l,r,rc[i],mid+1,sr);
29    }
30    void solve(istream &cin){
31        cin>>n>>q;
32        sz=rsc=0;
33        forn(i,1,n+1) cin>>a[i];
34        rt[++rsc]=++sz;
35        built(rt[1],1,n);
36        while(q--){
37            ll op; cin>>op;
38            if(op==3){
39                ll k; cin>>k;
40                rt[++rsc]=++sz;
41                lc[rt[rsc]]=lc[rt[k]],rc[rt[rsc]]=rc[rt[k]];
42                ↪ rc[rt[k]],t[rt[rsc]]=t[rt[k]];
43            }
44            else if(op==1){
45                ll k,a,x; cin>>k>>a>>x;
46                rt[k]=pst(a,x,rt[k],1,n);
47            }
48            else{
49                ll k,a,b; cin>>k>>a>>b;
50                cout << query(a,b,rt[k],1,n) << '\n';
51            }
52        }
53    }
54 }

```

splay-tree.cpp

```

1 constexpr ll mxN=2e5+3;
2 ll n,m,rt,t[mxN][2],cnt[mxN],val[mxN],f[mxN],lz[mxN],val;
3 string s;
4 ll dir(ll x){return t[f[x]][1]==x;}
5 void push_down(ll x){
6     if(lz[x]){
7         swap(t[x][0],t[x][1]);
8         if(t[x][0]) lz[t[x][0]]^=1;
9         if(t[x][1]) lz[t[x][1]]^=1;
10        lz[x]=0;
11    }
12 }
13 void push(ll x){cnt[x]=cnt[t[x][0]]+1+cnt[t[x][1]],vals[x]=vals[t[x][0]]+val[x]+vals[t[x][1]];}
14 void rotate(ll x){
15     ll y=f[x],z=f[y],d=dir(x);
16     t[y][d]=t[x][d^1],t[x][d^1]=y;
17     if(t[y][d]) f[t[y][d]]=y;
18     if(z) t[z][dir(y)]=x;
19     f[y]=x,f[x]=z,push(y),push(x);
20 }
21 void splay(ll &z,ll x){

```

```

22     ll w=f[z];
23     for(ll y;(y=f[x])!=w;rotate(x)){
24         if(f[y]!=w) rotate(dir(x)==dir(y)?y:x);
25     }
26     z=x;
27 }
28 void build(){
29     rt=0;
30     forn(i,1,n+3){
31         t[i][0]=rt;
32         if(rt) f[rt]=i;
33         rt=i;
34     }
35     splay(rt,1);
36 }
37 void loc(ll &z,ll k){
38     ll x=z;
39     for(push_down(x);cnt[t[x][0]]+1!=k;push_down(x)){
40         if(k>cnt[t[x][0]])
41             ↪ k-=cnt[t[x][0]]+1,x=t[x][1];
42         else x=t[x][0];
43     }
44     splay(z,x);
45 }
46 void opt(ll l,ll r){
47     loc(rt,l);
48     loc(t[rt][1],r-1+2);
49     ll x=t[t[rt][1]][0];
50     lz[x]^=1;
51 }
52 ll query(ll l,ll r){
53     loc(rt,l),loc(t[rt][1],r-1+2);
54     ll x=t[t[rt][1]][0],y=vals[x];
55     return y;
56 }
57 void solve(istream &cin){
58     cin >> n >> m;
59     forn(i,2,n+2) cin >> val[i];
60     build();
61     forn(i,0,m){
62         ll op,l,r; cin >> op >> l >> r;
63         if(op==1) opt(l,r);
64         else cout << query(l,r) << '\n';
65     }
66 }

```

6 Not that important

EPS.cpp

```

1 #include <cmath>
2 const double EPS = 1e-9;
3
4 bool eq(double a, double b) {
5     return fabs(a - b) < EPS;
6 }
7
8 bool lt(double a, double b) {
9     return a < b - EPS;
10 }
11
12 bool le(double a, double b) {
13     return a < b + EPS;
14 }

```

Gauze Elimination.cpp

```

1 // a[N][N] 是增广矩阵
2 int gauss()
3 {
4     int c, r;
5     for (c = 0, r = 0; c < n; c++)
6     {

```



```

7     int t = r;
8     for (int i = r; i < n; i ++ )    //
    ↪ 找到绝对值最大的行
9         if (fabs(a[i][c]) > fabs(a[t][c]))
10             t = i;
11
12     if (fabs(a[t][c]) < eps) continue;
13
14     for (int i = c; i <= n; i ++ ) swap(a[t][i],
    ↪ a[r][i]);    // 将绝对值最大的行换到最顶端
15     for (int i = n; i >= c; i -- ) a[r][i] /= a[r][c];
    ↪ // 将当前行的首位变成1
16     for (int i = r + 1; i < n; i ++ )    //
    ↪ 用当前行将下面所有的列消成0
17         if (fabs(a[i][c]) > eps)
18             for (int j = n; j >= c; j -- )
19                 a[i][j] -= a[r][j] * a[i][c];
20
21     r ++ ;
22 }
23
24 if (r < n)
25 {
26     for (int i = r; i < n; i ++ )
27         if (fabs(a[i][n]) > eps)
28             return 2; // 无解
29     return 1; // 有无穷多组解
30 }
31
32 for (int i = n - 1; i >= 0; i -- )
33     for (int j = i + 1; j < n; j ++ )
34         a[i][n] -= a[i][j] * a[j][n];
35
36 return 0; // 有唯一解
37 }

```

Joseph.cpp

```

1 // 回傳 0-indexed 的倖存者位置
2 ll josephus(ll n, ll k) { // O(n)
3     ll res = 0;
4     for (ll i = 1; i <= n; ++i) res = (res + k) % i;
5     return res;
6 }
7 ll josephus2(ll n, ll k) { // O(k log n), k<=n 時用; k>n
    ↪ 落回迴圈版避免深遞迴爆棧
8     if (n == 1) return 0;
9     if (k == 1) return n - 1;
10    if (k > n) {
11        ll res = 0;
12        for (ll i = 2; i <= n; ++i) res = (res + k) % i;
13        return res;
14    }
15    ll res = josephus2(n - n / k, k);
16    res -= n % k;
17    if (res < 0)
18        res += n; // mod n
19    else
20        res += res / (k - 1); // 还原位置
21    return res;
22 }

```

7 New

Convex_hull_trick.cpp

```

1 // 特定題解法: 斜率=(y2-y1)/m 是整數除法(負數向零截斷),
    ↪ 查詢x需單調遞增
2 class line{
3     public:
4         ll m,c;
5         line(){

```

```

6         line(ll m,ll c):m(m),c(c){}
7         friend bool operator<(const line &a,const
    ↪ line &b){
8             return a.m<b.m || (a.m==b.m &&
    ↪ a.c>b.c);
9         }
10        ll cal(ll x){return m*x+c;}
11    };
12    constexpr ll mxN=1e5+1;
13    ll n,m,id=0,st[mxN];
14    vc<line> ln;
15    bool check(ll i,ll j,ll k){
    ↪ //__int128防溢位(c差*m差可到1e18以上)
16        return
    ↪ (__int128)(ln[i].c-ln[k].c)*(ln[j].m-ln[i].m)<
    ↪ (__int128)(ln[i].c-ln[j].c)*(ln[k].m-ln[i].m);
17    }
18    void solve(istream &cin){
19        cin >> n >> m;
20        forn(i,0,n){
21            ll y1,y2; cin >> y1 >> y2;
22            ln.emp(line{(y2-y1)/m,y1});
23        }
24        sort(all(ln)),ln.resize(unique(all(ln),[](auto
    ↪ a,auto b){return a.m==b.m;})-ln.bg());
25        forn(i,0,ln.size()){
26            while(id>1 && check(st[id-1],st[id],i))
    ↪ id--;
27            st[++id]=i;
28        }
29        ll ptr=1;
30        forn(i,0,m+1){
31            while(ptr+1<=id && ln[st[ptr+1]].cal(i)>ln[
    ↪ st[ptr]].cal(i))
    ↪ ptr++;
32            cout << ln[st[ptr]].cal(i) << " \n"[i==m];
33        }
34    }

```

Ray_casting.cpp

```

1 class Point{
2     public:
3         ll x,y;
4         Point(){}
5         Point(ll x,ll y):x(x),y(y){}
6         friend Point operator-(const Point
    ↪ &a,const Point &b){return
    ↪ {a.x-b.x,a.y-b.y};}
7         friend ll operator*(const Point &a,const
    ↪ Point &b){return a.x*b.x+a.y*b.y;}
8         friend ll operator^(const Point &a,const
    ↪ Point &b){return a.x*b.y-a.y*b.x;}
9    };
10    ll n,m;
11    vc<Point> p;
12    bool on(Point &a,Point &b,Point &c){
13        if((b-a)^(c-a)) return false;
14        return 0<=((b-a)*(c-a)) &&
    ↪ ((b-a)*(c-a))<=((b-a)*(b-a));
15    }
16    void query(Point &x){
17        ll ok=0;
18        forn(i,0,n){
19            if(on(p[i],p[(i+1)%n],x)){ cout <<
    ↪ "BOUNDARY" << '\n';return;}
20            Point &a=p[i],&b=p[(i+1)%n];
21            if(a.y>x.y != b.y>x.y){
22                ll sg=(x-a)^(b-a);
23                if(b.y>a.y==(sg<0)) ok^=1;
24            }
25        }
26        cout << (ok&1?"INSIDE":"OUTSIDE") << '\n';
27    }

```


block_cut_tree.cpp

```

1  constexpr ll mxN=1e5+1;
2  constexpr ll mxB=20;
3  ll n,m,q,st[mxN],id=0,mark[mxN]{},dfn[mxN]{},low[mxN],tim=
    ↪ 0,nid[mxN],pa[mxN<<1][mxB],d[mxN<<1];
4  vc<ll> adj[mxN];
5  vc<vc<ll>> cmp,t;
6  void tarjan(ll u=1,ll p=0){
7      dfn[u]=low[u]=++tim,st[++id]=u;
8      for(auto&v:adj[u]){
9          if(v==p) continue;
10         if(dfn[v]) low[u]=min(low[u],dfn[v]);
11         else{
12             tarjan(v,u);
13             low[u]=min(low[u],low[v]);
14             if(low[v]>=dfn[u]){
15                 mark[u]=(dfn[u]>1 ||
16                     ↪ dfn[v]>2);
17                 cmp.emp(vc<ll>{u});
18                 while(cmp.back().back()!=v)
19                     ↪ cmp.back().emp(st[id--]);
20             }
21         }
22     }
23     void dfs(ll u=1,ll p=0,ll lv=1){
24         d[u]=lv,pa[u][0]=p;
25         for(auto&v:t[u]){
26             if(v!=p) dfs(v,u,lv+1);
27         }
28     }
29     void built(){
30         ll x=0;
31         t.emp(vc<ll>{});
32         forn(i,1,n+1) if(mark[i])
33             ↪ nid[i]=++x,t.emp(vc<ll>{});
34         for(auto&a:cmp){
35             ll y=++x;
36             t.emp(vc<ll>{});
37             for(auto&u:a){
38                 if(mark[u]) t[nid[u]].emp(y),t[y]
39                     ↪ emp(nid[u]);
40                 else nid[u]=y;
41             }
42         }
43         dfs();
44         forn(j,1,mxB) forn(i,1,t.size())
45             ↪ pa[i][j]=pa[pa[i][j-1]][j-1];
46     }
47     ll lca(ll a,ll b){
48         if(d[a]<d[b]) swap(a,b);
49         forr(i,mxB-1,0) if(d[a]-(1<<i)>=d[b]) a=pa[a][i];
50         if(a==b) return a;
51         forr(i,mxB-1,0) if(pa[a][i]!=pa[b][i])
52             ↪ a=pa[a][i],b=pa[b][i];
53         return pa[a][0];
54     }
55     void solve(){
56         cin >> n >> m >> q;
57         forn(i,0,m){
58             ll u,v; cin >> u >> v;
59             adj[u].emp(v),adj[v].emp(u);
60         }
61         tarjan();
62         built();
63         while(q--){
64             ll a,b,c; cin >> a >> b >> c;
65             cout << ((mark[c] &&
66                 ↪ query(nid[a],nid[b],nid[c])) || a==c
67                 ↪ || b==c?"NO":"YES") << '\n';

```

```

    }
}

```

cross_dot_product.cpp

```

1  //A=(x1,y1),B=(x2,y2)
2  //A cross B = ABsin x = x1*y2-y1*x2
3  //A dot B = AB cos x = x1*x2+y1*y2

```

dec_string_to_bitset.cpp

```

1  // 10進位string(只含0-9, 長度<=500) 轉 bitset 二進位,
    ↪ 支援先減一個小常數
2  // decSub(s,x): 回傳 s-x (x>=0, 需保證 s>=x), 0(len)
3  // decToBits(s): 10進string -> bitset, b[0]為LSB, 0(len^2)
4  // bitsToDec(b): bitset -> 10 進 string, 驗證/輸出用
5  // 500 位 10 進 1661 bits, mxB=2048 夠用
6  constexpr ll mxB=2048;
7  string decSub(string s,ll x){
8      for(ll i=s.size()-1;i>=0&&x;i--){
9          ll d=(s[i]-'0')-x%10; x/=10;
10         if(d<0){d+=10; x++;}
11         s[i]=d+'0';
12     }
13     ll p=0; while(p+1<(ll)s.size()&&s[p]=='0') p++;
14     return s.substr(p);
15 }
16 bitset<mxB> decToBits(string s){
17     bitset<mxB> b;
18     for(ll i=0;s!="0";i++){
19         b[i]=(s.back()-'0')&1;
20         ll r=0;
21         for(char &c:s){ll d=r*10+(c-'0');
22             ↪ c='0'+d/2; r=d&1;}
23         ll p=0; while(p+1<(ll)s.size()&&s[p]=='0')
24             ↪ p++;
25         s=s.substr(p);
26     }
27     return b;
28 }
29 string bitsToDec(bitset<mxB> b){
30     string s="0";
31     ll hi=mxB-1; while(hi>0&&!b[hi]) hi--;
32     forr(i,hi,0){
33         ll c=b[i];
34         forr(j,(ll)s.size()-1,0){ll
35             ↪ d=(s[j]-'0')*2+c; s[j]='0'+d%10;
36             ↪ c=d/10;}
37         if(c) s=char('0'+c)+s;
38     }
39     return s;
40 }
41 void solve(istream &cin){
42     string s; ll k;
43     cin>>s>>k; // 10 進 string 與 要減的常數
44     bitset<mxB> b=decToBits(decSub(s,k));
45     ll hi=mxB-1; while(hi>0&&!b[hi]) hi--;
46     forr(i,hi,0) cout<<b[i]; // MSB->LSB 印出 2 進位
47     cout<<'\n';
48 }

```

dominator_tree.cpp

```

1  constexpr ll mxN=1e5+1;
2  ll n,m,tim=0,id[mxN],sd[mxN],pa[mxN],f[mxN],lb[mxN],et[mxN]
    ↪ {}},ret[mxN];
3  vc<ll> adj[mxN],radj[mxN],t[mxN],st;
4  void dfs(ll u=1){
5      et[u]=++tim,ret[tim]=u;
6      id[tim]=sd[tim]=lb[tim]=pa[tim]=tim;
7      for(auto&v:adj[u]){

```

```

8         if(!et[v]) dfs(v),f[et[v]]=et[u];
9         radj[et[v]].emp(et[u]);
10    }
11 }
12 ll find(ll u,ll x=0){
13     if(u==pa[u]) return x?-1:u;
14     ll v=find(pa[u],x+1);
15     if(v<0) return u;
16     if(sd[lb[u]]>sd[lb[pa[u]]]) lb[u]=lb[pa[u]];
17     pa[u]=v;
18     return x?v:lb[u];
19 }
20 void built(){
21     vc<ll> arr[n+1];
22     forr(i,n,1){
23         for(auto&v:radj[i])
24             ↪ sd[i]=min(sd[i],sd[find(v)]);
25         if(i>1) arr[sd[i]].emp(i);
26         for(auto&v:arr[i]){
27             ll x=find(v);
28             if(sd[x]==sd[v]) id[v]=sd[v];
29             else id[v]=x;
30         }
31         if(i>1) pa[i]=f[i];
32     }
33     forn(i,2,n+1){
34         if(id[i]!=sd[i]) id[i]=id[id[i]];
35         t[ret[i]].emp(ret[id[i]]),t[ret[id[i]]].emp
36             ↪ p(ret[i]);
37     }
38 }
39 void dfs_ans(ll u=1,ll p=0){
40     st.emp(u);
41     if(u==n){
42         sort(all(st));
43         cout << st.size() << '\n';
44         for(auto&v:st) cout << v << ' '; cout <<
45             ↪ '\n';
46         exit(0);
47     }
48     for(auto&v:t[u]){
49         if(v==p) continue;
50         dfs_ans(v,u);
51     }
52     st.pop_back();
53 }
54 void solve(istream &cin){
55     cin>>n>>m;
56     forn(i,1,m+1){
57         ll u,v; cin>>u>>v;
58         adj[u].emp(v);
59     }
60     dfs();
61     built();
62     dfs_ans();

```

extra.cpp

```

1 //__builtin_popcountll
2 //__builtin_parityll
3 //__builtin_clzll
4 //__builtin_ctzll
5 // mt19937 rng(chrono::steady_clock::now().time_since_epoch
6     ↪ h().count());
7 // const ll M = 991831889;
8 // const ll C = uniform_int_distribution<ll>(0.1 * M, 0.9
9     ↪ M)(rng);

```

fft.cpp

```

1 // 兩模數NTT+CRT: 結果係數必須 < 998244353*1004535809
2     ↪ (~1.0027e18), 否則需三模
3 // sz上限2^21 (n+m+2 <= 2097152);
4     ↪ solve()讀n+1,m+1個係數(n,m是次數)
5 class FFT{
6     public:
7         ll mod,root,root_inv,maX,M,M_inv;
8         FFT(ll a,ll b,ll c,ll d,ll e,ll
9             ↪ f):mod(a),root(b),root_inv(c),maX(d),M
10             ↪ (e),M_inv(f){
11             ll fsp(ll a,ll m){
12                 ll ans=1;
13                 while(m){
14                     if(m&1) ans=(ans*a)%mod;
15                     a=(a*a)%mod,m>>=1;
16                 }
17                 return ans;
18             }
19             void fft(ll sz,vc<ll> &arr,ll inv){
20                 for(ll i=1,j=0;i<sz;++i){
21                     ll bit=sz>>1;
22                     ↪ for(;j&bit;bit>>=1)
23                     ↪ j^=bit; j^=bit;
24                     if(i<j)
25                         ↪ swap(arr[i],arr[j]);
26                 }
27                 for(ll l=2;l<=sz;l<<=1){
28                     ll wlen=inv?root_inv:root;
29                     for(ll i=1;i<maX;i<<=1)
30                         ↪ wlen=(wlen*wlen)%mod;
31                     for(ll i=0;i<sz;i+=l){
32                         ll w=1;
33                         for(ll
34                             ↪ j=0;j<l/2;++j){
35                             ll x=arr[i+j]
36                             ↪ +j],y=
37                             ↪ (w*arr
38                             ↪ [i+j+l
39                             ↪ /2])%m
40                             ↪ od;
41                             arr[i+j]=(
42                             ↪ x+y<mo
43                             ↪ d?x+y:
44                             ↪ x+y-mo
45                             ↪ d);
46                             arr[i+j+l/
47                             ↪ 2]=(x-
48                             ↪ y>0?x
49                             ↪ -y:x-y
50                             ↪ +mod);
51                             w=(w*wlen)
52                             ↪ %mod;
53                         }
54                     }
55                 }
56                 if(inv){
57                     ll n_1=fsp(sz,mod-2);
58                     forn(i,0,sz){
59                         arr[i]=(arr[i]*n_1
60                             ↪ )%mod;
61                     }
62                 }
63             }
64         };
65         constexpr ll MM=1002772198720536577;
66         //
67         //g=3
68         //998244353=119*2^23 + 1
69         //g^c=15311432
70         //inv=469870224
71         //M1=1004535809
72         //M1^-1=332747959
73         //
74         //1004535809=479*2^21 + 1
75         //g^c=702606812

```

```

53 //inv=700146880
54 //M2=998244353
55 //M2^-1=669690699
56 ll mul(ll a,ll b){
57     __int128 aa=a,bb=b,cc;
58     cc=(aa*bb)%MM;
59     return (ll)cc;
60 }
61 void solve(){
62     FFT fft1(998244353,15311432,469870224,1<<23,100453
        ↪ 5809,332747959);
63     FFT fft2(1004535809,702606812,700146880,1<<21,9982
        ↪ 44353,669690699);
64     //ll x=(fft1.M*fft1.M_inv)%fft1.mod;
65     //ll y=(fft2.M*fft2.M_inv)%fft2.mod;
66     //err(x,y);
67     ll n,m;
68     cin >> n >> m;
69     vc<ll> a(n+1),b(m+1);
70     for(auto &v:a) cin >> v;
71     for(auto &v:b) cin >> v;
72     ll sz=1;
73     for(;sz<a.size()+b.size();sz<=1);
74     a.resize(sz),b.resize(sz);
75     vc<ll> A=a,B=b;
76     fft1.fft(sz,a,0);
77     fft1.fft(sz,b,0);
78     forn(i,0,sz) a[i]=(a[i]*b[i])%fft1.mod;
79     fft1.fft(sz,a,1);
80
81     fft2.fft(sz,A,0);
82     fft2.fft(sz,B,0);
83     forn(i,0,sz) A[i]=(A[i]*B[i])%fft2.mod;
84     fft2.fft(sz,A,1);
85
86     forn(i,0,n+m+1) cout <<
        ↪ (mul(a[i],fft1.M*fft1.M_inv) +
        ↪ mul(A[i],fft2.M*fft2.M_inv))%MM << "
        ↪ \n"[i==n+m];
87 }

```

fwf.cpp

```

1 constexpr ll mxB=21;
2 ll a[1<<mxB]{};
3 void fwf(ll *arr,ll type){
4     for(ll m=1;m<(1<<mxB);m<=1){ //m=半塊距離, 跑滿
        ↪ 1..2^(mxB-1) 共mxB層
5         for(ll i=0,k=m<<1;i+m<(1<<mxB);i+=k){
6             forn(j,0,m){
7                 ll x,y; x=arr[i+j],y=arr[i
                    ↪ +j+m];
8                 arr[i+j]=x+y,arr[i+j+m]=x-
                    ↪ y;
9                 if(type) arr[i+j]/=2,arr[i
                    ↪ +j+m]/=2;
10            }
11        }
12    }
13 }
14 void solve(istream &cin){
15     ll n,pfx,mx;
16     cin>>n;
17     mx=pfx=0;
18     a[0]=1;
19     forn(i,0,n){
20         ll x; cin>>x;
21         pfx^=x,a[pfx]++,mx=max(mx,a[pfx]);
22     }
23     fwf(a,0);
24     forn(i,0,1<<mxB) a[i]=a[i]*a[i];
25     fwf(a,1);
26     ll res; res=0;
27     // forn(i,0,17) cout << a[i] << " \n"[i==16];
28     forn(i,0,1<<mxB) res+=(!i?mx>1:a[i]>0);
29     cout << res << '\n';

```

```

30     forn(i,0,1<<mxB) if(!i?mx>1:a[i]) cout << i << '
        ↪ '; cout << '\n';
31 }
32

```

kmp-automaton.cpp

```

void kmp(){
    forn(i,1,m){
        ll j; j=k[i-1];
        while(j>0 && s[j]!=s[i]) j=k[j-1];
        if(s[j]==s[i]) k[i]=j+1;
    }
}

void solve(istream &cin){
    cin>>n>>s;
    m=s.length();
    kmp();
    //forn(i,0,m){err(i,k[i]);}
    forn(i,0,m+1){
        forn(j,0,26){
            if(i>0 && (s[i]-'A')!=j)
                ↪ nex[i][j]=nex[k[i-1]][j];
            else nex[i][j]=i+(s[i]-'A'==j);
        }
    }
    // forn(i,0,m+1){forn(j,0,26){err(i,j,nex[i][j]);}}
    dp[0][0][0]=1;
    forn(i,0,n){
        forn(f,0,2){
            forn(j,0,m+1){
                forn(c,0,26){
                    // err(i,j,c,nex[j]
                    ↪ ][c],dp[i][j]);
                    ll nf; nf=f|(nex[j]
                    ↪ ][c]==m);
                    dp[i+1][nex[j][c]]
                    ↪ [nf]=(dp[i+1][
                    ↪ nex[j][c]][nf]
                    ↪ +dp[i][j][f])%
                    ↪ mdl1;
                }
            }
        }
    }
    ll res; res=0;
    forn(i,0,m+1) res=(res+dp[n][i][1])%mdl1;
    cout << res << '\n';
}

```

li_chao_tree.cpp

```

class line {
public:
    ll m,c;
    line(){m=0,c=-inf-1;}
    line(ll m,ll c):m(m),c(c){}
    ll cal(ll x){return m*x+c;}
};

constexpr ll mxN=1e5+5;
ll n;
line t[mxN<<2];

void insert(line x,ll l,ll r,ll i,ll sl,ll sr){
    if(sr<l || r<sl || l>r) return;
    ll mid=(sl+sr)>>1;
    if(l<=sl && sr<=r){ //if extended otherwise remove
        ↪ this condition
        bool lf=t[i].cal(sl-1)<x.cal(sl-1),mf=t[i]
            ↪ .cal(mid-1)<x.cal(mid-1),rf=t[i].cal(s
            ↪ r-1)<x.cal(sr-1);
        if(mf) swap(x,t[i]);
        if(sl==sr) return;
    }
}

```

```

18         if(lf^mf) insert(x,l,r,i<<1,sl,mid);
19         if(rf^mf) insert(x,l,r,i<<1|1,mid+1,sr);
20         return;
21     }
22     insert(x,l,r,i<<1,sl,mid);
23     insert(x,l,r,i<<1|1,mid+1,sr);
24 }
25 ll query(ll x,ll i,ll sl,ll sr){
26     ll mid=(sl+sr)>>1;
27     if(sl==sr) return t[i].cal(x-1);
28     if(x<=mid) return
29         ↪ max(t[i].cal(x-1),query(x,i<<1,sl,mid));
30     ↪ max(t[i].cal(x-1),query(x,i<<1|1,mid+1,sr));
31 }
32 void solve(){
33     cin >> n;
34     forn(i,0,n){
35         ll op;
36         cin >> op;
37         if(op==1){
38             ll a,b,l,r;
39             cin >> a >> b >> l >> r;
40             line x[a,b];
41             insert(x,l+1,r+1,1,1,mxN-4);
42         }
43         else{
44             ll x;
45             cin >> x;
46             x=query(x+1,1,1,mxN-4);
47             if(x>-inf-1) cout << x << '\n';
48             else cout << "NO" << '\n';
49         }
50     }

```

manacher.cpp

```

1 ll dp[mxN][2],n,res[mxN];
2 void solve(istream &cin){
3     cin >> s,n=s.length();
4     ll l1,l2,r1=-1,r2=-1;
5     forn(i,0,n) res[i]=1;
6     forn(i,0,n){
7         if(r1>i) dp[i][1]=min(r1-i,dp[l1+r1-i][1]);
8         if(r2>i)
9             ↪ dp[i][0]=min(r2-i,dp[l2+r2-i-1][0]);
10        while(i+dp[i][1]<n && i-dp[i][1]>=0 &&
11            ↪ s[i+dp[i][1]]==s[i-dp[i][1]])
12            ↪ res[i+dp[i][1]]=max(res[i+dp[i][1]],2*
13            ↪ dp[i][1]+1,dp[i][1]);
14        while(i+dp[i][0]+1<n && i-dp[i][0]>=0 &&
15            ↪ s[i+dp[i][0]+1]==s[i-dp[i][0]])
16            ↪ res[i+dp[i][0]+1]=max(res[i+dp[i][0]+1],2*
17            ↪ dp[i][0]+1,dp[i][0]);
18        if(i+dp[i][1]>r1)
19            ↪ l1=i-dp[i][1],r1=i+dp[i][1];
20        if(i+dp[i][0]>r2)
21            ↪ l2=i-dp[i][0]+1,r2=i+dp[i][0];
22    }
23    forn(i,0,n) cout << res[i] << " \n"[i==n-1];
24 }

```

shoelace.cpp

```

1 //consecutive point p[i] (0<=i<n)
2 //注意: 輸出是「2倍面積」(鞋帶公式沒除2); 2A =
3     ↪ sum(p[i]^p[i+1])
4 //座標 |<=1e9 時 2A<=8e18 勉強不溢位, 更大要 __int128
5 void solve(istream &cin){
6     ll n,res=0;
7     cin>>n;
8     forn(i,1,n+1){

```

```

        cin>>a[i];
        if(i>1) res+=a[i-1]^a[i]; //cross product
        if(i==n) res+=a[n]^a[1];
    }
    cout << abs(res) << '\n';
}

```

suffix_automaton.cpp

```

1 // 靜態陣列約4.5MB(64MB限制小心);
2     ↪ 多次呼叫sam()時sz要歸零(solve有做)
3 constexpr ll mxN=1e5+1;
4 class state{
5     public:
6         ll len,link,next[26];
7
8     state(){}
9 };
10 ll n,sz;
11 state t[mxN<<1];
12 void sam(const string &s){
13     memset(t,0,sizeof(t));
14     t[0].link=-1;
15     ll last,cur; last=0;
16     for(auto&v:s){
17         cur++;sz;
18         t[cur].len=t[last].len+1;
19         ll p; p=last;
20         while(p!=-1 && !t[p].next[v-'a'])
21             ↪ t[p].next[v-'a']=cur,p=t[p].link;
22         if(p!=-1) t[cur].link=t[p].link;
23         else{
24             ll q; q=t[p].next[v-'a'];
25             if(t[p].len+1==t[q].len) t[cur].link=q;
26             else{
27                 ll clone; clone++;sz;
28                 t[clone].len=t[p].len+1,t[clone].link=t[q].
29                 ↪ .link;
30                 memcpy(t[clone].next,t[q].next,sizeof(t[q].
31                 ↪ .next));
32                 while(p!=-1 && t[p].next[v-'a']==q)
33                     ↪ t[p].next[v-'a']=clone,p=t[p].link;
34                 t[q].link=t[cur].link=clone;
35             }
36         }
37         last=cur;
38     }
39 }
40 void solve(istream &cin){
41     string s;
42     cin>>s;
43     n=s.length(),sz=0;
44     sam(s);
45     vc<ll> res(n+2,0);
46     forn(i,1,sz+1)
47         ↪ res[t[t[i].link].len+1]++,res[t[i].len+1]--;
48     partial_sum(all(res),res.bg());
49     forn(i,1,n+1) cout << res[i] << " \n"[i==n];
50 }

```

suffixarray_and_lcp.cpp

```

1 constexpr ll mxN=1e5+5;
2 ll sa[mxN],nsa[mxN],c[mxN],nc[mxN],cnt[mxN],lcp[mxN];
3 void sfa(string s){
4     s+="$";
5     ll n=s.length();
6     memset(cnt,0,sizeof(cnt)); //cnt有殘留,
7     ↪ 不清第二次呼叫必錯
8     forn(i,0,n) cnt[s[i]]++; forn(i,1,256)
9     ↪ cnt[i]+=cnt[i-1];
10    forr(i,n-1,0) sa[--cnt[s[i]]]=i; c[sa[0]]=0;
11    ↪ forn(i,1,n)
12    ↪ c[sa[i]]=c[sa[i-1]]+(s[sa[i]]!=s[sa[i-1]]);

```

```

9      for(ll h=0;(1<<h)<n;h++){
10          forn(i,0,n) nsa[i]=((sa[i]-(1<<h))%n+n)%n;
11          fill(cnt,cnt+c[sa[n-1]]+1,0);
12          forn(i,0,n) cnt[c[nsa[i]]]++;
13          ↪ forn(i,1,c[sa[n-1]]+1)
14          ↪ cnt[i]+=cnt[i-1];
15          forr(i,n-1,0) sa[--cnt[c[nsa[i]]]]=nsa[i];
16          nc[sa[0]]=0;
17          forn(i,1,n){
18              pll pre={c[sa[i-1]],c[(sa[i-1]+(1<<h))%n]},cur={c[sa[i]],c[(sa[i]+(1<<h))%n]};
19              ↪ ]+(1<<h))%n];
20              nc[sa[i]]=nc[sa[i-1]]+(cur!=pre);
21          }
22          swap(c,nc);
23      }
24  }
25  void Lcp(const string &s){
26      ll n=s.length(),k=0;
27      forn(i,0,n){
28          if(c[i]>n-1){k=0; continue;}
29          ll j=sa[c[i]+1];
30          for(;i+k<n && j+k<n && s[i+k]==s[j+k];k++);
31          lcp[c[i]]=k;
32          if(k) k--;
33      }
34  }

```

xor_basis.cpp

```

1  // 值需 < 2^31; 值域到 1e18 時 mxB 改 60、迴圈上限改 59
2  constexpr ll mxN=2e5+1;
3  constexpr ll mxB=31;
4  ll n,a[mxN],b[mxB]{};
5  void built(){
6      forn(i,1,n+1){
7          ll x; x=a[i];
8          forr(j,30,0){
9              if((x>>j)&1){
10                 if(!b[j]){b[j]=x; break;}

```

```

          else x^=b[j];
        }
    }
}

void solve(istream &cin){
    cin>>n;
    forn(i,1,n+1) cin>>a[i];
    built();
    ll res; res=0;
    forr(i,30,0){
        if(res>>i&1^1) res^=b[i];
    }
    cout << res << '\n';
}

```

zfunction_and_kmp.cpp

```

constexpr ll mxN=1e6+1;
vc<ll> zf(const string &s){
    ll n,l,r; n=s.length(),l=r=-1;
    vc<ll> z(n,0);
    forn(i,1,n){
        if(r>i) z[i]=min(z[i-1],r-i);
        while(i+z[i]<n && s[i+z[i]]==s[z[i]])
            ↪ z[i]++;
        if(i+z[i]>r) r=i+z[i],l=i;
    }
    return z;
}

vc<ll> kmp(const string &s){
    ll n; n=s.length();
    vc<ll> k(n,0);
    forn(i,1,n){
        ll j; j=k[i-1];
        while(j>0 && s[i]!=s[j]) j=k[j-1];
        if(s[i]==s[j]) k[i]=j+1;
    }
    return k;
}

```